MOAI RESEARCH · OKTOBER 2026

Ideen für ObjectFlow und ManMap

Was SAP RAP, Business Central, die Java-Plattformen für modellgetriebene Entwicklung, die Open-Source-ERPs, das Persistenz-Ökosystem, Event- und Prozess-Frameworks, moderne Typsysteme und Low-Code-Produkte deklarieren, das MoWare-Anwendungen noch von Hand bauen, und was die beiden Sprachen davon aufnehmen könnten.

Zusammengestellt aus sieben Recherchenotizen in `research/research_notes/Ideas for ObjectFlow and ManMap/`. Quelle: `research/reports/Ideas for ObjectFlow and ManMap.md`. Deutsche Übersetzung.

Für WM,
der nie aufhört, nach neuen Ideen zu suchen.

„Der beste Weg, eine gute Idee zu haben, ist, viele Ideen zu haben.“

Linus Pauling

Lebenszyklus, Events und Graph-Persistenz in die MoWare-DSLs heben

Über SAP RAP, Business Central, die Java-Plattformen für modellgetriebene Entwicklung, die Open-Source-ERPs, das Java-Persistenz-Ökosystem, die Event- und Prozess-Frameworks, moderne Typsysteme und die Low-Code-Produkte hinweg kehren dieselben wenigen Konzeptfamilien als *Deklarationen* wieder, die MoWare-Anwendungen heute noch von Hand bauen: Regeln am Domänenmodell mit deklarierten Auslösern und einer festen Speicherphase, eine Übergangstabelle am Status-Konzept, Domain Events mit garantiertem Versand nach dem Commit, ein persistierter Prozess oder eine Arbeitswarteschlange mit Retry- und Fristsemantik, eine deklarierte Komposition, aus der das Speichern „erst Eltern, dann Kinder“ und das Löschen verwaister Kinder generiert wird, Lesemodelle, deren SELECT aus einer deklarierten Gestalt abgeleitet wird, und Fakten pro Property (Nullbarkeit, Constraints, dimensionsbehaftete Zahlen), die aus einer einzigen Deklaration in generiertes Java, das Schema, die Serdes und jeden API-Vertrag fließen. Drei Laufzeitfähigkeiten tragen die meisten dieser Ideen: die Session als *phasierte* Unit of Work mit benannten Einhängepunkten (sie besitzt bereits die Transaktion, daher lassen sich Determinations, Validations, Event-Registry-Zeilen und Outbox-Zeilen in den Check-in generieren), ein *persistierter Scheduler mit Arbeitswarteschlange* (Fristen, Timer, Retries, Incidents, Idempotenzschlüssel und Prozessinstanzen brauchen ihn alle, und die flüchtige Batch-Inbox ist heute das Einzige in diesem Bereich) sowie ein *graphbewusster Generator* für ManMap (Kompositionen, Load-Graphen, abgeleitete Views, Aggregate und Historisierung sind mechanisch, sobald der FK-Graph deklariert ist). Die externen Belege bestätigen außerdem mehrere bewusste Auslassungen in MoWare, vor allem kein Lazy Loading, keine implizite Kaskade und kein Event Sourcing. Die Empfehlung lautet daher, deklarierte, sichtbare, nicht transitive Varianten dieser Mechanismen hinzuzufügen, statt ORM-Voreinstellungen zu kopieren. Die Ideen unten sind nach Themen gruppiert; jede Idee nennt die externen Systeme, die sie umsetzen, skizziert die Form in ObjectFlow oder ManMap und beschreibt kurz, wie MoWare-Entwickler sie heute von Hand bauen. Grundlage dafür sind `docu/objectflow.md` (OF), `docu/manmap.md` (MM), `docu/moware-werkbank.md` (WB), `docu/dataux.md` (DUX), `conventions/` (CONV) und die beiden Problemprotokolle (PR1, PR2), wie in der Baseline-Notiz zusammengefasst. Ein eigenes Kapitel ordnet jedes wiederkehrende handgeschriebene Muster den Ideen zu, die es aufnehmen würden, und ein Schlusskapitel listet die kleinen Optionen und Checker-Regeln.

Wie die Belege zu lesen sind

Die Notizen stützen sich überwiegend auf Primärdokumentation (Herstellerhandbücher, Spezifikationen, Quellcode, JEPs), einige Bereiche beruhen jedoch auf Sekundärmaterial und sind entsprechend zu gewichten. Das SAP Help Portal und die SAP-Community-Seiten waren nicht erreichbar, daher stammen die RAP-Fakten aus SAPs eigenem Learning-Kurs, GitHub-Cheat-Sheets, Entwickler-Tutorials und SAP Press. Das RAP Background Processing Framework (bgPF) und die RAP-Änderungsbelege beruhen nur auf Suchauszügen, und die Behauptung, Managed RAP „behandle Änderungsbelege automatisch“ (Praktiker-Blog), wird von einer SAP-Community-Antwort widersprochen, laut der ein zusätzliches Save plus ein API-Aufruf nötig ist. Änderungsbelege sind in RAP daher als handgeschrieben zu betrachten (SAP cheat sheet 08). Die Entwickler- und Hilfeseiten von Salesforce waren nicht erreichbar; die Order of Execution ist aus einem Sekundär-Blog und einer Suchzusammenfassung wiedergegeben, die sich darin widersprechen, wo eigene Validation Rules relativ zu Before-Save-Flows liegen (salesforcegeek). Die iDempiere-Wikiseiten waren

gesperrt, daher sind mehrere iDempiere-Fakten Such-Snippets, während die Fakten auf Code-Ebene aus dem GitHub-Quellcode stammen. Die Entfernung der Workflow-Engine aus Odoo (v10/v11) ist nur durch Forenbeiträge belegt, und der Erscheinungsmonat von Odoo 19 wird von einer Quelle mit September, von einer anderen mit Oktober angegeben. Hibernates „keine API, um soft-gelöschte Zeilen einzuschließen“ beruht auf einem Forenbeitrag. Die Flyway-Namenskonventionen stammen aus Leitfäden Dritter, weil die offizielle Seite 404 lieferte. Fowlers `allocate` für Money beruht auf Sekundärdarstellungen von PoEAA. Der Vergleich Camunda gegen Code-first ist Hersteller- und Beratermaterial. Schließlich ist jede Einstufung BIG/SMALL in den Notizen eine Einschätzung der Rechercheure, keine belegte Tatsache; dieser Report verwendet sie als Vorschlag.

Thema A. Regeln und Typen am Domänenmodell

A1. Deklarierte Constraints und Verfeinerungstypen an Business Properties

Die Idee ist, dass eine Business Property oder ein benannter Werttyp mit einer einzigen Property ihren Constraint trägt (Längenbereich, Zahlenbereich, Nachkommastellen, Muster, nicht leer, eindeutig), sodass eine Deklaration einen validierenden Konstruktor, das Schema (`VARCHAR(n)`, `DECIMAL(p,s)`, `CHECK`, `UNIQUE`, `NOT NULL`), Serdes und Schlüsselwörter im API-Vertrag sowie UI-Metadaten erzeugt. Typisierte Sprachen lösen das mit Verfeinerungstypen (Refinement Types): Iron für Scala 3 schreibt `Int :| Positive` oder `String :| Alphanumeric`, prüft Literale zur Compile-Zeit und verfeinert Laufzeitwerte über `refineEither`/`refineValidation` (letzteres sammelt alle Fehler), und verwendet den Constraint in JSON-, Datenbank- und Konfigurationsintegrationen wieder (Iron). Kotlin-Value-Classes tragen einen `init { require(...) }`-Block, sodass der Typ nur über einen einzigen validierten Weg konstruiert werden kann (Kotlin inline classes); das Rust-Buch zeigt dasselbe mit einem `Guess` mit privatem Feld und prüfendem Konstruktor, sodass nachgelagerte Funktionen „keine zusätzlichen Prüfungen mehr bräuchten“ (Rust book ch09-03); F#-Single-Case-Unions umhüllen Primitive, damit eine E-Mail-Adresse nicht mit einer Postleitzahl verwechselt wird (F# single-case DUs). Alexis Kings „Parse, don't validate“ benennt die Entwurfsregel: Eine Prüfung soll einen präziseren Typ zurückgeben, damit der Nachweis erhalten bleibt, und soll einmal an der Grenze stattfinden statt als „Shotgun Parsing“ über den Verarbeitungscode verstreut (Parse, don't validate). Die Low-Code-Plattformen tun dasselbe deklarativ: Mendix-Validierungsregeln an Entitäten sind eine geschlossene Liste (required, unique, equals, range, regulärer Ausdruck, maximale Länge), die beim Commit in konfigurierter Reihenfolge geprüft wird, Eindeutigkeit zuletzt, weil sie die Datenbank braucht; alle fehlschlagenden Meldungen werden gesammelt und je Aufrufer ausgegeben (Seitenmeldung, Microflow-Fehler oder Java-Exception) (Mendix validation rules). Salesforce-Validation-Rules sind eine Wahr/Falsch-Formel plus eine an ein bestimmtes Feld gebundene Meldung, mit `ISNEW()`, `ISCHANGED(field)` und `PRIORVALUE(field)`, um eine Regel auf einen Übergang einzuschränken (Trailhead validation rules, best practices). Causeways `@Property` trägt `maxLength`, `regexPattern`, `optionality` und `mustSatisfy`-Spezifikationen, und eine Abweichung zwischen `optionality` und `@Column(nullable)` lässt den Start fehlschlagen (Causeway @Property). OpenAPI 3.1 und JSON Schema tragen `minLength`, `pattern`, `minimum` und `exclusiveMinimum` im Vertrag (OpenAPI 3.0 to 3.1).

In ObjectFlow sähe das als Constraint-Klauseln am Property-Typ aus, zum Beispiel `string(1..60)`, `BigDecimal(12,2) >= 0`, `pattern '...'`, dazu ein Root-Konzept `value type CustomerNo = string(8) pattern '[0-9]{8}'`, das als Property-Typ in Entities, Value Objects und DTOs verwendbar ist und von ManMap ohne Schemaänderung auf den zugrunde liegenden Spaltentyp abgebildet wird. Generierte Value Objects und Werttypen wären Java-Records, deren kompakter Konstruktor die Prüfung enthält

(JEP 395); ein Literal, das einer eingeschränkten Property zugewiesen wird, ließe sich im MPS-Editor prüfen, analog zu Irons Literalprüfung zur Compile-Zeit. Die Grenze DTO zu Entity ist der Ort, an dem die eine Konvertierung vom Benutzer korrigierbare Preconditions auslöst, sodass nachgelagerter Entity-Code keine erneuten Prüfungen trägt.

In MoWare heute. Die Property-Optionen umfassen bereits `LENGTH` (min/max, das auch die Pflichteingabe in der UI steuert) und `RANGE` (int und BigDecimal), und ManMap ergänzt `SIZE`, `NOTNULL`, `UNIQUE` als Schemahinweise (OF § „Business Properties“; MM § „Felder, Schlüssel und Optionen“). Alles darüber hinaus lebt in Preconditions innerhalb von Service Methods, wird je Anwendungsfall wiederholt und ist für ManMap, Serdes und jede API unsichtbar. Die Konventionen verlangen DTO-Eingabeprüfungen als CheapCode, die „Service-Prüfungen nie ersetzen“, und PR1 P-033 hält fest, dass Validierungen doppelt modelliert wurden, weil `LENGTH` bereits die Pflichteingabe steuerte (CONV `mw-domaenenmodell_v1.md` § „Eingabeprüfungen und fachliche Prüfungen“; PR1 P-033). Ein fachlicher Typ mit einer einzigen Property, etwa eine Rechnungsnummer, ist ein nackter `string`, sodass nichts verhindert, dass eine Kundennummer übergeben wird, wo eine Rechnungsnummer erwartet ist.

A2. Entity-Regeln mit deklarierten Auslösern und phasiertem Speichern

Beide großen kommerziellen Plattformen hängen Regeln an die Entität und deklarieren, *wann* sie laufen, statt wo sie aufgerufen werden. SAP RAP unterscheidet Determinations (Werte ableiten) von Validations (zurückweisen), jeweils mit Auslösebedingungen `create; update; delete; field a, b;` und einer Phase: `on modify` läuft sofort im Transaktionspuffer und kann mehrfach feuern, `on save` läuft genau einmal in der Speichersequenz (`finalize` für Determinations, `check_before_save` für Validations); eine fehlgeschlagene Validation „weist den gesamten Puffer zurück“; die Ausführungsreihenfolge unter Determinations und Validations ist „nicht festgelegt“ (SAP cheat sheet 36, SAP Learning determinations, SAP Learning validations). SAP Press beschreibt die Wahl zwischen on modify und on save als Frage der „Abhängigkeit, nicht der Kosten“ und warnt, dass ein als Auslöser verwendetes Feld bei jeder Änderung dieses Feldes feuert (SAP Press decision model); Praktiker-Blogs berichten, dass Teams „zu viel Logik in Determinations packen, sodass sie sich wie Mini-Workflows verhalten“, und dass das Vermischen von Determinations und Validations „subtile Fehler verursacht“ (dev.to RAP deep dive). RAPs Speichersequenz benennt die Phasen (`finalize`, `check_before_save`, `adjust_numbers`, `save`, `cleanup`) mit einem Point of No Return, und der Strict Mode macht Phasenverstöße (Datenbankschreibzugriffe außerhalb des Late Save, Events außerhalb von Save) zu Prüfungen (SAP cheat sheet 08). Business Central legt dasselbe vor: `OnInsert`/`OnModify`/`OnDelete` laufen vor der Datenbankoperation und brechen sie bei Fehler ab, das feldbezogene `OnValidate` läuft nach Typ- und Eigenschaftsprüfungen, und die dokumentierte Reihenfolge um ein Delete ist Before-Event, Tabellentrigger, globaler Trigger, Datenbankoperation, After-Event. Der Haken ist, dass Trigger nur laufen, wenn der Aufrufer `RunTrigger = true` übergibt, eine bekannte Quelle stillschweigend übersprungener Logik (BC OnModify, BC OnValidate, BC event types). Die ERP-Familie läuft auf dasselbe Vokabular zu: iDempieres `ModelValidator` zählt `TYPE_BEFORE_NEW`/`CHANGE`/`DELETE` und `AFTER_*` plus sechzehn Beleg-Zeitpunkte auf (ModelValidator.java); metasfreshs `@ModelChange(types, ifColumnsChanged, afterCommit)` macht den Hook spaltenbewusst und lässt ihn nach dem Commit laufen (ModelChange.java); Odoos `@api.constrains('name','description')` läuft nur, wenn ein aufgeführtes Feld unter den geschriebenen Werten ist (Odoo ORM); Mendix-Event-Handler sind Before/After x Create/Commit/Delete-Microflows, die einen Boolean zurückgeben, wobei „Fehler auslösen, wenn der Microflow false liefert“ einen Before-Commit-Handler wie eine native Validierung wirken lässt (Mendix event handlers); Jmix' `EntityChangedEvent` trägt `AttributeChanges` mit alten Werten und unterscheidet einen Before-Commit-Listener (eine Exception rollt alles zurück) von einem After-Commit-Listener (Jmix entity events); Salesforce legt eine dokumentierte Speicher-Pipeline fest, von Validation Rules über Before-Trigger, Speichern,

After-Trigger, Roll-ups bis zum Commit, mit der Warnung, dass Workflow-Feldaktualisierungen Trigger erneut auslösen, die Validierung aber nicht (salesforcegeek, sekundär).

In ObjectFlow ist die natürliche Form eine an einer Entity deklarierte Regelfamilie: `determination setDueDate on modify { create; field invoiceDate, paymentTerms }` und `validation datesConsistent on save { create; update; field beginDate, endDate }`. Der Generator leitet die Methodenrümpfe ab, und die Session führt sie beim Check-in in einer festen, benannten Phasenreihenfolge aus: Determinations, dann Validations, die alle Verstöße in einen Problembericht sammeln, dann `save with`-Anweisungen, dann Reaktionen nach dem Commit (siehe B3). Der Regelrumpf bleibt imperatives BaseLanguage mit Zugriff auf den vorherigen Wert und Änderungsflags pro Property. Der Checker erhält günstige, wertvolle Regeln nach dem Vorbild des RAP-Strict-Mode: kein ManMap-Aufruf `CHECKIN/DELETE` oder `STATEMENT` innerhalb einer Determination oder Validation, keine Event-Auslösung vor dem Commit und Zyklenerkennung über Feldauslöser, damit Determinations sich nicht endlos gegenseitig speisen.

In MoWare heute. Preconditions gehören zu einer Service Method und laufen, wenn diese Methode aufgerufen wird. Das empfohlene Muster ist „Fakten laden, ein `validation`-Block, dann mutieren“, und die Sprache erzwingt weder, dass Prüfungen vor Änderungen stattfinden, noch dass sie überhaupt stattfinden (OF § „Preconditions, Validation, Guards und Exceptions“; WB § „Grundprinzipien“ Punkt 6). Nichts in der Sprache bindet eine Regel an eine Entity oder an die Änderung einer Property; ein Service, der den Prüfaufruf vergisst, persistiert einfach den inkonsistenten Graphen, denn „Properties einer Entity sind auch von Commands und anderen Services aus setzbar“ (CONV `mw-domaenenmodell_v1.md` § „Service und Aggregat“). Dirty-Tracking existiert pro Entity, legt dem fachlichen Code aber keinen alten Wert pro Property offen (OF § „Dirty-Tracking, Read-only und unveränderliche Werte“).

A3. Nullbarkeit pro Property als deklarierte Tatsache

Kotlin macht Abwesenheit zum Typ: `String` kann kein null halten, `String?` schon, und der Compiler erzwingt `?.`, `?:` oder einen Smart Cast; die dokumentierten verbleibenden NPE-Quellen sind `!!`, Initialisierungslecks und Java-Interop mit „Plattformtypen“ unbestimmter Nullbarkeit (Kotlin null safety). Rust hat kein null; `Option<T>` und `T` sind verschiedene Typen, sodass `x + y` mit `y: Option<i8>` nicht kompiliert (Rust book ch06-01). Für generiertes Java macht JSpecify 1.0 (Juli 2024, vier Annotationen, mit dem Versprechen, sich nie inkompatibel zu ändern) `@NonNull String` zu einem Subtyp von `@Nullable String`; `@NullMarked` an einem Package macht nicht annotierte Typen nicht-null, und `@NullUnmarked` dient als Ausweg für die schrittweise Einführung (JSpecify user guide, release). Oracles eigener API-Hinweis beschränkt `Optional` auf Rückgabetypen von Methoden (Optional Javadoc), und Michael Ernst argumentiert, `Optional` „macht Ihren Code nicht korrekter“, sondern verwandle lediglich eine NPE in eine `NoSuchElementException`; er bevorzugt `@Nullable`-Annotationen mit einem statischen Checker (Ernst). In Verträgen ist GraphQL standardmäßig nullbar, mit `!` für nicht-null und Nullbarkeit pro Listenebene (GraphQL schema), während OpenAPI 3.1 null als `type: [string, "null"]` ausdrückt (OpenAPI 3.0 to 3.1).

Für ObjectFlow bedeutet das eine Markierung `mandatory/optional` (oder `T?`) pro Business Property mit Voreinstellungen auf Ebene des Modells oder Moduls, sodass bestehende Modelle „unmarkiert“ bleiben und neue sich bewusst dafür entscheiden, genau Kotlins Migrationspfad über Plattformtypen. Generierter Code trägt JSpecify-Annotationen, die null-sicheren Operatoren `:eq:/:ne:` werden auf nicht nullbaren Properties überflüssig, `status switch` braucht einen `null`-Zweig nur bei nullbaren Status-Properties, ManMap leitet `NOT NULL` ab, und Serdes sowie jeder API-Vertrag leiten `required/!` ab. Die bestehende Regel „ein Ja/Nein ist ein Status“ sollte bleiben, damit nullbare Booleans nicht wieder eingeführt werden.

In MoWare heute. Nullbarkeit ist ein Satz von Konventionen pro Typ: `string` ist nie null (SQL-`NULL` wird zu `""`), `int` beginnt bei 0, `LocalDate`, `DateTime`, Value Object und Referenzen beginnen bei `null`, und ein Status beginnt mit seinem `ON_CREATION`-Element. Gleichheitsoperatoren unterscheiden sich je Typ, `==` auf einem Value-Object-Schlüssel ist ein Checker-Fehler, und `LocalDate == LocalDate` wurde stillschweigend zu einem Referenzvergleich kompiliert (OF § „Null-Werte in Datenstrukturen“; PR1 P-023). Ein Status, der berechtigterweise „noch nicht gesetzt“ ist, braucht `ALLOW_NULL_PERSISTANCE` und einen Konstruktor, der die `ON_CREATION`-Voreinstellung auf `null` zurücksetzt (OF § „Status“; PR2 P-011, P-034), und `optional` in ManMap-Filtern behandelt `0`, `null` und `""` als ungesetzt, wobei `-1` und `""` ausdrücklich ausgeschlossen sind (MM § „Spezifikum - Filterausdrücke und gemappte Felder“).

A4. Status mit Nutzdaten pro Zustand

Java 17 und 21 haben algebraische Datentypen in den Mainstream gebracht: ein `sealed interface` mit `permits`, Records als Produkthälfte und ein `switch`, dessen Vollständigkeit der Compiler prüft, der Record-Komponenten dekonstruiert und `MatchException` wirft, wenn zur Laufzeit ein neuer erlaubter Typ auftaucht. Das JEP rät, bei Enum-Switches `default` wegzulassen, damit neue Konstanten auffallen, und behandelt `case null` als expliziten Zweig (JEP 409, JEP 441, JEP 440). Scala-3-Enums mischen einfache Werte mit parametrisierten Fällen und warnen bei unvollständigen Matches (Scala 3 ADTs, Scala tour); Rust-Enum-Varianten tragen unterschiedliche Daten (`V4(u8,u8,u8,u8)` gegenüber `V6(String)`) (Rust book ch06-01); der F#-Essay darüber, ungültige Zustände nicht darstellbar zu machen, ersetzt „E-Mail optional, Post optional“ durch `EmailOnly | PostOnly | EmailAndPost`, sodass der ungültige Fall „keines von beiden“ nicht konstruiert werden kann (F# illegal states). Auf der Leitung sind das GraphQL-Unions und OpenAPI-`oneOf` mit einem `discriminator` (OAS latest). Die erweiterbaren Enums von Business Central fügen eine Strategie-Variante hinzu: Ein Enum kann ein Interface per `implements` umsetzen und jeden Wert an eine Codeunit binden, sodass `iAddressProvider := sendTo` anhand des Werts dispatcht (BC interfaces).

Der Status in ObjectFlow ist bereits die Summenhälfte; die Ergänzung sind Nutzdaten pro Element, zum Beispiel `status Shipment { Planned; Shipped(carrier: Carrier, trackingNo: string); Delivered(at: DateTime, signedBy: string) }`, generiert als Sealed Interface plus Records mit einem dekonstruierenden `status switch` ohne `default`. Felder, die nur in einem Zustand Bedeutung haben, sind dann keine nullbaren Spalten mit impliziter Invariante mehr. ManMap hat dann zwei mechanische Abbildungen: eine Status-Spalte plus nullbare Spalten pro Zustand mit generiertem `CHECK`, oder eine Nebentabelle pro Zustand. Jedes Status-Element an eine Service-Implementierung zu binden (BCs `implements`) ist ein kleinerer Folgeschritt.

In MoWare heute. Ein Status ist ein reichhaltiges Enum mit Persistenzwert, Beschreibungen, Farbe und einem vollständigen `status switch` (OF § „Status“; § „Statuswerte mit `status switch` behandeln“), aber zustandsspezifische Daten leben in gewöhnlichen nullbaren Properties der Entity, deren Gültigkeit per Konvention vom aktuellen Status abhängt. Der Generator erzeugt eine Vollständigkeitsprüfung nur, wenn `default` fehlt, und ein `null`-Status braucht einen Zweig `default: // and null`.

A5. Geld und Mengen als dimensionsbehaftete Typen

Fowlers Position ist, dass „Geld in keiner gängigen Programmiersprache tatsächlich ein erstklassiger Datentyp ist“, dass eine Money-Klasse das Addieren verschiedener Währungen abfängt, das Runden zentralisiert, nie Gleitkommazahlen verwenden darf und eine Operation `allocate` braucht, die einen Betrag nach Anteilen aufteilt, ohne Cents zu verlieren (Fowler, Money; Allokation über Sekundärdarstellungen Verraes). Sein Quantity-Entwurf verallgemeinert das auf Betrag plus Einheit, mit Zurückweisung unsinniger Additionen und expliziter Umrechnung, und hält

fest, dass Währungsumrechnung einen Zeitbezug braucht, weshalb automatische Umrechnung meist unzulässig ist (Fowler, Quantity). F#-Maßeinheiten bieten Dimensionsprüfung zur Compile-Zeit (`float<m/s>` kann nicht zu `float<m>` addiert werden), werden aber zur Laufzeit gelöscht, sodass Einheiten nicht ausgegeben werden können (F# units of measure); Kotlin-Value-Classes liefern einen eigenen Typ ohne Kosten, außer wo geboxt wird (Kotlin inline classes). Causeway bietet einen Werttyp-Vertrag auf Framework-Ebene (parsen, darstellen, Voreinstellungen, zerlegen) und zusammengesetzte Werte, die als `@Embeddable` persistiert werden (Causeway value types).

Für ObjectFlow sind zwei getrennte Typen angebracht statt eines generischen `quantity<unit>`: ein Typ `money`, der Betrag und Währung zur Laufzeit trägt (die F#-Löschung greift nicht, weil die Währung aus der Datenbank kommt), mit Addition und Vergleich nur bei gleicher Währung, Multiplikation mit Skalar, `allocate` und expliziter Umrechnung über ein datiertes Kursobjekt; und ein `quantity<unit>` für physikalische Einheiten. ManMap erhält eine kanonische Abbildung auf zwei Spalten (Betrag `DECIMAL` plus Währung `CHAR(3)`), mit der Option, nur den Betrag abzubilden, wenn die besitzende Entity einen festen Währungskontext deklariert; das ist die Speicherwahl, die Fowler offenlässt.

In MoWare heute. Geld ist ein nackter `BigDecimal` mit Formatstring und Literalen wie `13.44bd`; `double` und `float` sind verboten (OF § „Literale für Datum, Zeitpunkt und Dezimalzahl“). Währung, Rundung und anteilige Verteilung werden je Anwendung von Hand geschrieben, und nichts verhindert, einen Nettobetrag zu einem Prozentsatz zu addieren.

A6. Benannte, kombinierbare Spezifikationen und Entscheidungstabellen

Evans und Fowler definieren eine Specification als Prädikatobjekt mit `isSatisfiedBy`, das „wie ein Kandidat zu prüfen ist“ vom Kandidaten trennt und zugleich Auswahl, Validierung und Bau nach Auftrag dient; zusammengesetzte Spezifikationen lassen Programmierer Blätter und Kombinatoren liefern, und `remainderUnsatisfiedBy` kann dem Benutzer sagen, warum eine Spezifikation nicht erfüllt wurde (Evans & Fowler, Specifications). Spring Datas `Specification` lässt sich mit `and`/`or` kombinieren und verwendet seit 4.0 dasselbe Prädikat für Select, Update und Delete (Spring Data Specifications). Odoo-Domains sind eine einzige Ausdruckssprache für Suche, Record Rules, Automatisierungsauslöser und Window Actions, auswertbar sowohl in SQL als auch im Speicher über `filtered_domain` (Odoo ORM); metasfreshs `IQueryFilter` akzeptiert ebenso ein Modell im Speicher oder wird zu SQL gerendert (IQueryBuilder.java). Für tabellarische Regeln legen DMN-Entscheidungstabellen die Semantik bei Mehrfachtreffern mit Hit Policies fest: Unique, Any, First, Rule order und Collect mit SUM/MIN/MAX/COUNT, wobei bei Unique mehrere Treffer ein Fehler sind (Camunda hit policies). Easy Rules ist im Wartungsmodus und Drools lebt unter dem inkubierenden Apache KIE, was für einen Tabellen-*Ausdruck* in der DSL spricht statt für eine eingebettete Engine (Easy Rules, Apache KIE).

In ObjectFlow würde ein Root `specification OverdueInvoice on Rechnung { ... }` mit Kombination über `and`/`or`/`not` als Precondition dienen (`precondition OverdueInvoice(rechnung)`), als Filter in einem gemappten `where` (zu SQL kompiliert, wo das Prädikat über gemappte Felder ausdrückbar ist) und als Erklärungsquelle für die vom Benutzer korrigierbare Meldung. Ein Ausdruck `decision table` mit Eingaben, Ausgaben und einem Hit-Policy-Enum deckt „Rabatt, Priorität oder Genehmiger aus mehreren Attributen ableiten“ ab, wobei Unique zur Testzeit geprüft wird.

In MoWare heute. Eine Precondition ist an eine Service Method gebunden und wird je Methode einmal ausgedrückt; dasselbe fachliche Prädikat taucht als ManMap-`where`-Ausdruck oder SQL-Fragment in einem `<Thema>LeseRepo` wieder auf, und als `#Meta`-Freigabelogik in einer Scope-Funktion (OF § „Preconditions, Validation, Guards und Exceptions“; MM § „Gemappte Abfragen mit

`QueryFromMap`“; OF § „Scopes“). Ableitungen aus mehreren Attributen sind verschachtelte `if`-Anweisungen in Service Methods.

Thema B. Lebenszyklus, Status und Prozess

B1. Eine deklarierte Übergangstabelle am Status

Keine der untersuchten Geschäftsplattformen hat einen deklarativen Zustandsautomaten, das ist also echtes Neuland. RAP leitet erlaubte Operationen aus dem Status in einem imperativen `FOR INSTANCE FEATURES`-Handler ab (`%features-%update = COND #( WHEN OverallStatus = accepted THEN fc-o-disabled ...)`) und kodiert Übergänge als Actions (RAP100 dynamic feature control); Business Central verwendet ein Status-Enum mit Release/Reopen-Codeunits und eine tabellengesteuerte Workflow-Engine, die etwa acht Registrierungs-Subscriber braucht (BC status, BC workflow walkthrough); iDempiere berechnet gültige Actions je Status in einer rund 350 Zeilen langen, handkodierten Fallunterscheidung pro Tabelle (DocumentEngine.java); Axelor schützt Übergangs-Buttons mit `showIf="statusSelect == 1"` im View-XML (AOS SaleOrder views); Causeway, OpenXava und Jmix haben gar kein Konstrukt dafür. OFBiz kommt am nächsten mit Zeilen `StatusValidChange(statusId, statusIdTo, conditionExpression, transitionName)`, die Services vor einer Statusänderung prüfen (OFBiz entitymodel.xml, OrderServices.java). Die Zustandsautomaten-Bibliotheken liefern das Vokabular: Stateless4j mit `permit`, `permitIf(guard)`, `permitInternal` (ohne Exit/Entry), `permitReentry`, `ignore`, `permitDynamic`, `onEntry`/`onExit`, `substateOf` (StateConfiguration.java); Spring Statemachine mit `timerOnce` für zeitgesteuerte Übergänge und `StateMachineRuntimePersister` für Persistenz (Spring Statemachine); XState mit reinen, benannten Guards, die über `and/or/not` kombiniert werden, Arrays geschützter Übergänge mit ungeschütztem Default und verzögerten `after`-Übergängen (XState guards, transitions). metasfresh ergänzt eine nützliche Zusicherung: `processEx(document, action, expectedDocStatus)` prüft nach der Action den resultierenden Status (IDocumentBL.java).

In ObjectFlow erlaubt ein `transitions`-Block an der Status-Deklaration, `from DRAFT on freigeben [guard] -> FREIGEGEBEN`, mit optionalen `entry`/`exit`-Actions pro Element und Gruppierung über `substate of`, dem Generator Folgendes abzuleiten: das Gerüst der Service Method mit ihrer Precondition „Status in {Quellen}“, die Menge der für den aktuellen Status freigegebenen Übergänge (als Ersatz für handgeschriebene `#Meta setElements`/`setEnabled`-Logik und `generally enabled`-Ausdrücke an Commands), eine Darstellung in DOT oder PlantUML, Prüfungen auf unerreichbare Zustände und Sackgassen sowie `run command`-Testzusicherungen auf Statuspfaden. Hierarchische Zustände decken „alle `IN_PROGRESS.*` sind stornierbar“ ab; parallele Regionen können entfallen, wie bei Stateless4j.

In MoWare heute. Die Konvention ist eine Service Method pro Übergang, benannt nach dem fachlichen Akt, mit Preconditions auf den aktuellen Status (CONV `mw-domaenenmodell_v1.md` § „Aggregate und Invarianten“). Die Menge der zulässigen Übergänge existiert nur implizit über diese Methoden verteilt, lässt sich für UI oder Tests nicht aufzählen, und jedes Command wiederholt die Freigabe in `generally enabled` oder einer Scope-Funktion. PR2 P-042 hält fest, dass eine an die echte Property gebundene Statuswechsel-Maske das Command zwingt, den alten Status um die Service-Prüfung herum zu merken und wiederherzustellen.

B2. Belegtyp mit Lebenszyklus und Buchen nur durch Anfügen

Die Compiere-Familie bettet ein festes Belegvokabular ein: `DocAction`-Status Drafted, In Progress, Approved, Completed, Closed, Voided, Reversed und Actions Prepare, Complete, Void, Close, Reverse-Correct, Reverse-Accrual, Re-activate, wobei `prepareIt()` und `completeIt()` den neuen Status zurückgeben und sechzehn Before/After-Hook-Zeitpunkte sie umgeben. `DocumentEngine.processIt` liest `DocStatus` unter einer Zeilensperre erneut und wirft `DocStatusChanged`, wenn eine andere Session ihn geändert hat, und eine Stornierung erzeugt einen über `Reversal_ID` verknüpften Gegenbeleg, statt zu löschen (DocAction Javadoc, DocumentEngine.java). Die Periodensteuerung ist ein abtrennbarer Constraint: `MPeriod.isOpen(ctx, dateAcct, docBaseType, orgId, forPosting)` entscheidet je Basisbelegart, Organisation und Action gegenüber Buchung (MPeriod.java). metasfresh ersetzte das Ableiten von Unterklassen durch ein `DocumentHandler`-SPI und ein typisiertes `DocStatus`-Enum mit Prädikatmethoden (DocumentHandler.java). Business Central strukturiert jede Transaktion als Journalzeilen, eine Check-Codeunit, eine Post-Line-Codeunit, die „der einzige Ort zum Einfügen von Sachposten“ ist, und unveränderliche Posten plus ein Register innerhalb einer Transaktion mit expliziten Tabellensperren. Microsoft selbst berichtet, dass Codeunit 12 vor dem Neuentwurf der Buchungsengine auf etwa 7.600 Zeilen angewachsen war (BC Gen. Jnl.-Post Line, posting engine). Die Buchhaltungsfakten in iDempiere sind eine regenerierbare Projektion nach Abschluss (`Doc.createFacts`; erneutes Buchen verwirft und erzeugt neu) (Doc Javadoc).

In ObjectFlow ist das eine Status-Variante `document status`, die das Lebenszyklusvokabular und die Hook-Slots vordefiniert (before/after prepare, complete, void, close, reverse), dazu einen Ergebnisstatus-Vertrag pro Action, die Zusicherung des erwarteten Status, ein erneutes Lesen des Status unter Sperre vor einer Action und eine Precondition-Familie `period control`, die nach Belegart, Datum, Organisation und Action gegliedert ist. Eine `GRAPH_OWNER_CMD`-Art, die nur anfügende Einträge erzeugt, mit ManMap-Mappings, die als `APPEND_ONLY` markiert sind, sodass der Checker `UPDATE`/`DELETE` darauf zurückweist, bildet das Hauptbuchmuster ab.

In MoWare heute. Jedes Aggregat implementiert seinen Lebenszyklus neu als Service Methods mit von Hand gewählten Status-Elementen, und aggregatübergreifende Wirkungen des Abschlusses (Lieferung anlegen, Posten buchen) werden in denselben Methoden oder in Commands kodiert. Nichts unterscheidet eine veränderliche Stammdaten-Entity von einem unveränderlichen gebuchten Posten, und nichts verhindert ein `delete with` darauf (MM § „Löschen mit `delete with`“; CONV `mw-domaenenmodell_v1.md`).

B3. Domain Events mit garantiertem Versand nach dem Commit und einer Publikations-Registry

Jede untersuchte Plattform besteht darauf, dass Benachrichtigungen erst nach dem Commit ausgelöst werden: RAP-Business-Events werden mit `RAISE ENTITY EVENT` im Late Save „nach dem Point of No Return“ ausgelöst und asynchron konsumiert (SAP Learning events); die externen Business Events von Business Central werden „erst an externe Systeme gesendet, nachdem die aktuelle Transaktion festgeschrieben ist“, und seine isolierten Events führen jeden Subscriber in einer eigenen Transaktion aus, sodass ein fehlschlagender Subscriber den Publisher nicht zurückrollen kann (BC business events, BC isolated events); OFBiz' SECA `global-commit-post-run` mit `statusId != oldStatusId` ist das Idiom für „nach dem Commit auf eine Zustandsänderung reagieren“ (order secas.xml); metasfreshs `afterCommit = true` warnt, dass Arbeit in einem solchen Hook eine eigene Transaktion braucht und Fehler nur protokolliert werden (ModelChange.java); Odoo-Automatisierungsregeln lösen bei `on_state_set`, `on_create_or_write` und datumsbasierten Bedingungen aus, mit Domain-Filtern vor und nach der Änderung (base_automation.py). Axon zeigt die beiden Versandarten, die eine DSL benennen muss: Ein Subscribing Processor läuft im Thread und in der Transaktion des Publishers, ein Streaming Processor läuft in einem eigenen Thread mit

persistiertem Tracking-Token, Retries von 1 bis 60 Sekunden und Dead-Letter-Queues. Axons zustandsgespeichertes Aggregat beweist, dass Events aus einer zustandspersistierten Entität ohne Event Sourcing publiziert werden können (Axon state-stored aggregates, event processors). Die Event Publication Registry von Spring Modulith ist die minimale zuverlässige In-Process-Form: Sie „schreibt Einträge für jeden [Listener] ... als Teil der ursprünglichen Geschäftstransaktion“, markiert sie als abgeschlossen, nachdem der Listener erfolgreich war, verfolgt Versuche und Status (PUBLISHED, PROCESSING, COMPLETED, FAILED, RESUBMITTED) und publiziert offene Einträge beim Neustart erneut; die eigene Dokumentation räumt ein, dass die native Externalisierung „kritische Funktionen vermissen lässt, die Entwickler von echten Outbox-Implementierungen erwarten würden“ (Spring Modulith events). Das eigentliche Transactional-Outbox-Muster speichert die ausgehende Nachricht in der Geschäftstransaktion und lässt ein Relay sie publizieren. Das garantiert „gesendet genau dann, wenn die Datenbanktransaktion festgeschrieben wird“, um den Preis von Duplikaten, sodass Konsumenten idempotent sein müssen; der genannte Nachteil, dass „der Entwickler das Publizieren vergessen könnte“, entfällt, wenn die DSL die Zeile schreibt (microservices.io outbox). Der durchgängige Rat lautet, Events ohne Event Sourcing einzuführen: Dudycz nennt CRUD-Systeme, Datensack-Module und unerfahrene Teams als schlechte Kandidaten und betont, „es ist keine Alles-oder-nichts-Entscheidung“ (Dudycz); Vernons Regeln „echte Invarianten in Konsistenzgrenzen modellieren“ und „außerhalb der Grenze Eventual Consistency verwenden“ sind das DDD-Argument für Events statt breiterer Sessions (Vernon Part II).

In ObjectFlow ist das ein Root `event RechnungFreigegeben(rechnungKey, betrag)`, eine `emit`-Anweisung in Service Methods und Handler-Deklarationen `on event RechnungFreigegeben (in session | after commit)` mit einer Reihenfolge- und Fehlerstrategie (protokollieren und fortfahren, weiterreichen, Dead-Letter). Die Session schreibt die Registry- oder Outbox-Zeilen in derselben Transaktion wie den Check-in, sodass „emit beim Commit“ eine garantierte Semantik ist; ein `consumer/producer pair` oder ein Relay leert die Outbox für externe Konsumenten. Die Testunterstützung folgt Modulith' `AssertablePublishedEvents`: Ein `run command` kann zusichern, dass ein Event mit bestimmten Eigenschaften publiziert wurde.

In MoWare heute. Domain Events existieren nicht; die Werkbank-Dokumentation sagt, der Stack „übernimmt aber nicht sämtliche DDD-Konzepte“ (WB § „Zentrale Konzepte der MoWare-Werkbank“). Die nächstliegenden Mechanismen sind `session queue next command`, das ein UI-Command nach dem Commit einplant, in Tests und Jobs ignoriert wird und „ersetzt jedoch keine allgemeine Job- oder Workflow-Engine“ (OF § „Command nach dem Commit einplanen“), sowie `TO_SESSION_OPS`-Service-Methods, die der Transaktion des Besitzers beitreten. Aggregatübergreifende Reaktionen werden im festschreibenden Command kodiert, und eine Regel über zwei Aggregate kollidiert mit der Konvention „keine Zyklen“ und wird dadurch gelöst, dass der Aufrufer Fakten übergibt (PR2 P-050). Eine Suche, die sich nach einer Bearbeitung aktualisiert, braucht einen handgeschriebenen `any cmd term`-Handler (PR2 P-044).

B4. Persistierte Prozessinstanzen, Sagas und Benutzeraufgaben

Camunda 8 persistiert den Prozesszustand als BPMN-Modell: Ein Job entsteht, wenn die Ausführung eine Aufgabe erreicht, wird von Job Workern mit At-least-once-Zustellung und expliziten Retries abgeholt, erschöpfte Retries lösen einen Incident aus, bis er behoben ist, und Benutzeraufgaben tragen Bearbeiter, Kandidatengruppen, Fälligkeitsdatum, Priorität, Formularbindung und Listener, während „die Prozessinstanz an dieser Stelle anhält und wartet“ (Camunda job workers, user tasks). Temporal macht beliebigen deterministischen Code dauerhaft, indem es eine Event-Historie wieder abspielt. Der Preis sind Determinismusregeln, patchbasierte Versionierung und harte Historiengrenzen von 51.200 Events oder 50 MB, die ein Continue-as-new erzwingen; sein Nachrichtenmodell trennt Signal (asynchrones Schreiben), Query (Lesen, ohne Historie) und Update (synchron, vor Annahme validiert) (Temporal workflow definition, limits, message passing).

Eventuates Saga-DSL ist klein genug zum Abschreiben: `step().withCompensation(reject).step().invokeParticipant(reserveCredit).step().invokeParticipant(approve)`, Vorwärtsschritte sequenziell, Kompensationen in umgekehrter Reihenfolge (Eventuate sagas); Microsofts Saga-Leitfaden ergänzt die Einteilung in Pivot- und wiederholbare Schritte sowie Gegenmaßnahmen wie die semantische Sperre, die buchstäblich ein als ausstehend markierter Statuswert ist (Microsoft saga). Axon-Sagas werden über eine Zuordnungs-Property mit `@StartSaga`/`@EndSaga` geroutet (Axon sagas). OutSystems startet einen Prozess deklarativ bei der Create-Action einer Entität und schließt eine menschliche Aktivität „bei einem `Create<Entity>`/`Update<Entity>`-Event mit Attributbedingungen“ (OutSystems Process, Human Activity); Mendix-Workflows tragen eine `$WorkflowContext`-Entität durch Benutzeraufgaben, Entscheidungen und Timer (Mendix workflows). Axelor und Jmix delegieren an BPMN-Engines (Flowable), iDempiere hat eine datendefinierte Engine im WfMC-Stil, und die Engine von Business Central ist aus Laufzeittabellen zusammengesetzt, ein warnendes Beispiel dafür, was passiert, wenn das Prozesskonzept nicht in der Sprache ist (BC workflow walkthrough). Der Vergleich Code-first gegen BPMN beruht auf Hersteller- und Beraterseiten, der strukturelle Punkt gilt aber: MPS gibt ObjectFlow bereits ein projektionales, grafisch darstellbares Modell, sodass das Argument eines gemeinsamen visuellen Modells für BPMN ohne zweites XML-Artefakt zu haben ist.

In ObjectFlow würde ein Root `process RechnungsFreigabe keyed by rechnungKey` die Ein-Schritt-Mechanismen ersetzen, mit geordneten Schritten (lokaler Service-Aufruf, Benutzeraufgabe für eine Rolle mit Fälligkeitsdatum, Warten auf ein Event oder eine Frist, Aufruf eines anderen Aggregats mit Kompensation), einer persistierten Schrittposition und typisierten Prozessdaten, einer Retry-Strategie pro Schritt sowie Startstrategien im Vokabular von Temporal (Duplikat zurückweisen, bestehende verwenden). Eine Benutzeraufgabe bildet auf ein bestehendes Command ab, das einer Rolle angeboten wird und dessen `FINAL OK_CONCLUSION` den Prozess fortsetzt; ein Warteschritt ist erledigt, wenn eine Entity einen deklarierten Status erreicht (B1) oder ein Event (B3) eintrifft. Die Position zu persistieren statt Code wieder abzuspielen, vermeidet Temporals Determinismus-Zeremonie. Laufende Instanzen überleben aber trotzdem den Code, der sie erzeugt hat, daher braucht das Einfügen von Schritten eine deklarierte Migrationsregel, wie es Camundas versionierte Deployments tun.

In MoWare heute. Mehrschrittige Abläufe werden als Successor-Commands ausgedrückt, die sich die Session teilen, den Commit des Vorgängers aufschieben, bis der Nachfolger erfolgreich ist, und Ketten ausdrücklich nicht unterstützen („Eine weitere Successor-Kette am Ziel-Command wird nicht unterstützt“), oder als `session queue next command`, das nur für die UI gilt (OF § „Successor-Commands“; § „Command nach dem Commit einplanen“). Alles, was Tage, Benutzer oder Neustarts überspannt, ist ein Status an der Entity plus ein `consumer/producer pair`, dessen Producer-Abfrage die Arbeit jedes Mal neu findet (DUX § „Batchjob mit `BatchJob Module`“).

B5. Zeit als Event-Quelle: Fristen, Timer und Zeitpläne

Drei Konstrukte kehren wieder. Fristen mit Instanzbezug: Axons `deadlineManager.schedule(Duration, "name", payload)` mit einem `@DeadlineHandler` im selben Aggregat oder in derselben Saga, stornierbar per ID oder per Scope und bewusst nicht als Event gespeichert (Axon deadline managers). Persistierte Timer innerhalb eines Prozesses: Temporal-Timer „werden persistiert“, kosten während des Wartens nichts, laufen von einer Sekunde bis zu mehreren Jahren und gelten als Mindestdauer (Temporal timers); XStates verzögerte `after`-Übergänge und Spring Statemachines `timerOnce` sind die Form im Zustandsautomaten (XState transitions, Spring Statemachine); Camunda-Benutzeraufgaben tragen `dueDate`/`followUpDate` (Camunda user tasks); die Odoo-Automatisierung bietet die Auslöser `on_time`, `on_time_created`, `on_time_updated`, die standardmäßig alle vier Stunden von einem Cron geprüft werden (Studio automation rules). Zeitpläne als erstklassige Objekte: Temporals Schedule hat eine eigene Identität, Überlappungsstrategien (Skip, BufferOne, BufferAll,

CancelOther, TerminateOther, AllowAll), ein Nachholfenster, Pausieren und Backfill (Temporal schedules); OutSystems-Timer deklarieren Zeitplan, ein Standard-Timeout von 20 Minuten, Retries, `Wake<Timer>` und laufen nie zweimal gleichzeitig (OutSystems Timer, ODC timers); die Job Queue von Business Central ergänzt Wiederholung, Protokollierung, eine Fehlerbehandlungs-Codeunit in einer neuen Session und eine Höchstzahl an Versuchen (BC job queue). Axon merkt an, dass eine einzelne Tabelle im Stil von db-scheduler die leichtgewichtige Persistenzoption ist und dass Quartz „in letzter Zeit wenig Aktivität gezeigt hat“ (Axon event schedulers).

Für ObjectFlow ist das eine `deadline`-Deklaration an einem Entity-Status oder Prozessschritt (`in AWAITING_PAYMENT after 14 days -> EXPIRED unless cancelled by zahlungEingegangen`), ein `wait until`-Schritt in Prozessen, ein Fälligkeitsdatum an Benutzeraufgaben und ein `schedule`-Objekt, das die nackten Optionen `CRON`/`DELAY` durch Überlappungsstrategie, Nachholfenster und Timeout ersetzt. Alle brauchen eine einzige persistierte Scheduler-Tabelle in der Laufzeitumgebung.

In MoWare heute. Zeit existiert nur als `CRON` und `DELAY` an einem `consumer/producer pair` in der Standardzeitzone der JVM. Es gibt kein Timeout pro Element, keine Überlappungsstrategie über „der Producer läuft nur, wenn kein Consumer arbeitet“ hinaus, und jede Frist ist eine Producer-Abfrage über Datumsspalten, die überfällige Arbeit bei jedem Lauf neu findet (DUX § „Kapitellandkarte: Batchoptionen“; § „Batchjob mit `BatchJob Module`“).

B6. Dauerhafte Arbeitswarteschlange mit Retry-Strategie, Incidents und Idempotenz

Die Workpackage-Queue von metasfresh persistiert Elemente und Parameter in `C_Queue_WorkPackage`/`C_Queue_Element`, bindet ein Paket an die umgebende Transaktion, sodass es erst nach dem Commit bereit wird (`bindToThreadInheritedTrx`, `markReadyForProcessingAfterTrxCommit`), übernimmt Mandanten-, Organisations- und Benutzerkontext vom Zeitpunkt des Einreihens und gibt Prozessoren `isAllowRetryOnError()` und eine Überspringen-Anforderung (IWorkPackageBuilder.java, IWorkpackageProcessor.java). OFBiz persistiert asynchrone Service-Aufrufe in `JobSandbox` mit `max-retry` und iCalendar-Wiederholung (OFBiz Service Engine Guide); Odoos Cron-Protokoll verarbeitet einen Batch, schreibt pro Batch über `_commit_progress` fest, überspringt nach drei aufeinanderfolgenden Fehlern und deaktiviert den Job nach fünf Fehlschlägen innerhalb von sieben Tagen (Odoo actions). Die Felder der Temporal-Retry-Strategie sind der kanonische Satz: Anfangsintervall 1 s, Backoff 2,0, maximales Intervall, maximale Versuche, nicht wiederholbare Fehlertypen und ein Gesamt-Timeout Schedule-to-Close, mit der Regel, dass Activities standardmäßig wiederholt werden und Workflows nicht (Temporal retry policies). Camunda benennt den Endzustand: Erschöpfte Retries lösen einen Incident aus, die Behebung wertet neu aus (Camunda job workers). Idempotenz ist die Folge von At-least-once überall; Stripes Vertrag (Client-Schlüssel, erstes Ergebnis wird gespeichert und bei Retry wiedergegeben, auch bei 500ern, abweichende Parameter werden zurückgewiesen, Schlüssel werden nach 24 Stunden bereinigt) ist die sauberste Spezifikation (Stripe idempotent requests), und Temporals Strategien zur Wiederverwendung von Workflow-IDs und zu Konflikten spezifizieren den idempotenten Prozessstart (Temporal workflow id).

In ObjectFlow erhielte das `consumer/producer pair` eine Option für eine persistierte Inbox mit Elementstatus, Versuchszähler und Incident-Zustand; eine wiederverwendbare `retry policy`-Deklaration mit den Feldern von Temporal, anhängbar an Consumer, Event-Handler und Prozessschritte; eine `incident`-Reaktion in der Exception-Strategie neben `READD_TO_INBOX`; und `idempotent by <Schlüsselausdruck>` an Commands und Service Methods, das eine Tabelle verarbeiteter Schlüssel generiert. Eine dritte Problemachse, vorübergehender gegenüber dauerhaftem technischem Fehler, erweitert die bestehende Trennung von Precondition und Guard und speist die Retry-Entscheidung.

In MoWare heute. „Die Inbox ist eine flüchtige In-Memory-Queue und keine persistente Jobwarteschlange“: Sie wird vor jedem Producer-Lauf geleert, geht beim Neustart verloren, ein Element wird vor der Verarbeitung entfernt und nur mit `READD_TO_INBOX` wieder hinzugefügt, ansonsten gibt es keinen Retry und kein Timeout pro Element. Consumer müssen idempotent sein, indem sie in einer Precondition erneut prüfen, ob die Arbeit noch nötig ist (DUX § „Batchjob mit `BatchJob Module`“; § „Exception-Strategien und Wiederanlauf“).

B7. Entwurf: eine persistierte, fortsetzbare, sperrbare Unit of Work (optional)

RAPs Draft ist der einzige Mechanismus ohne MoWare-Gegenstück: `with draft;` gibt jeder Entität eine Schatten-Entwurfstabelle, implizite Actions Edit, Activate, Discard, Resume sowie ein Prepare, das die On-save-Determinations und -Validations ausführt, eine exklusive Sperre auf die aktive Instanz, die „nicht an die ABAP-Session gebunden“ ist, mit einer Standardlebensdauer von 15 Minuten, gefolgt von einer optimistischen Phase, die durch ein Gesamt-ETag geschützt ist, und automatische Entfernung nach 28 Tagen Inaktivität. Es existiert, weil „zustandslose RESTful-Protokolle wie OData keine feste Session haben“ (SAP Learning draft, SAP cheat sheet 36). Die Kosten sind eine Schattentabelle pro Entität, `%is_draft` in jedem Schlüssel und scharfe Kanten mit Altschlüsseln, von denen in der Community berichtet wird. Causeways Command-Log zeichnet jede Top-Level-Action auf, und `BackgroundService` persistiert ein Command zur späteren Ausführung (Causeway command log).

Für ObjectFlow könnte eine `GRAPH_OWNER_CMD`-Option `DRAFTABLE` den Session-Graphen (Entities, Dirty-Zustand, ausstehende Session-Operationen) beim Benutzerabbruch mit Speichern in eine Entwurfstabelle serialisieren und ihn später mit den obigen Sperrregeln fortsetzen, unter Wiederverwendung der bestehenden Serdes. Das ist teuer und sollte optional bleiben.

In MoWare heute. Die Session liegt im Speicher und stirbt mit dem Command; `revert` wirkt nur im Speicher, `acquire locks` werden bei jedem Beenden freigegeben, und eine lange Bearbeitung, die durch einen Session-Verlust unterbrochen wird, beginnt wieder beim Datenbankstand (OF § „Revert beim Abbruch“; § „Pessimistische Sperren mit `acquire locks`“).

Thema C. Persistenz: was ManMap deklarieren könnte

C1. Deklarierte Komposition mit generiertem Graph-Speichern, Löschen verwaister Kinder und Baumversionierung

RAP unterscheidet eine Komposition von einer Assoziation über den Lebenszyklus: „Eine Kind-Entität … existiert nur zusammen mit ihrem Elternteil“, das Löschen eines verwalteten Elternteils löscht die Kompositionskinder, und die Wurzel ist Lock Master, Authorization Master und Besitzerin des `total etag` für den ganzen Baum, während Kinder `lock dependent by _Parent` deklarieren (SAP Learning composite BOs, SAP cheat sheet 36). Jmix' `@Composition` bedeutet, dass Kind und Besitzer gemeinsam in einer Transaktion bearbeitet und gespeichert werden, mit `@OnDelete(DENY|CASCADE|UNLINK)` und `@OnDeleteInverse` als deklarierten Löschstrategien (Jmix entities). Blaze-Persistence verlangt, dass aktualisierbare Views Kaskade und Entfernen verwaister Kinder explizit deklarieren und mit den Einstellungen der Entität übereinstimmen, sonst schlagen sie fehl (Blaze manual); die Operationen von Hibernates `StatelessSession` „kaskadieren nicht“ (Hibernate Introduction). Die Kritik an Standard-Kaskaden ist einhellig: Kaskadiertes REMOVE bei Many-to-many „könnte am Ende mehr löschen, als wir erwarten“ und wird oft erst im Nachhinein entdeckt (Vlad Mihalcea cascade types); Axelors One-to-many hat standardmäßig `orphanRemoval=true`, die

Gegenposition (Axelor models). Business Central kodiert das Löschen von Kindern explizit im `OnDelete` des Kopfes (BC data triggers).

Für ManMap ist die passende Form eine explizite, nicht transitive Option `composition` an einem `ListMapping` (und an einem `ReferenceMapping` für besessene Einzelkinder). Daraus erzeugt der Generator ein `save graph with <Mapping>(root)`, das den Elternteil speichert, den Schlüssel oder die Rückreferenz weitergibt, jedes Kind speichert und Kinder löscht, deren Schlüssel geladen wurden, die aber nicht mehr in der Liste sind; dazu ein `delete graph with` in der Reihenfolge „Kinder zuerst“, eine Strategie `DENY`/`UNLINK` für Referenzen von außerhalb des Aggregats und einen `OPTIMISTIC_LOCK`-Bereich über den ganzen Baum (RAPs Total ETag), sodass eine Kindänderung die TCN der Wurzel hochzählt. Einfache `ReferenceMapping`s bleiben aggregatübergreifende, schreibgeschützte Referenzen, wie es die Konventionen bereits verlangen.

In MoWare heute. `save with` speichert genau die Spalten eines Mappings; der Entwickler schreibt: „zuerst der Parent gespeichert, dann der Parent-Schlüssel beziehungsweise die Rückreferenz an die Kinder gesetzt und schließlich jedes Kind über sein eigenes `EntityMapping` gespeichert“, löscht Kinder explizit vor dem Elternteil und erkennt entfernte Kinder, indem er sich die geladene Liste merkt und sie in `FINAL OK_CONCLUSION` abgleicht, ein Workaround, der als kein moai-Thema dokumentiert ist (MM § „Speichern von Objektgraphen“; § „Löschen mit `delete with`“; PR2 P-041). Jedes Aggregat wiederholt das in seinem `<Aggregat>Repo.checkin`.

C2. Lesemodelle, deren SELECT und Joins aus einer deklarierten Gestalt folgen

Die vorherrschende Idee im Persistenz-Ökosystem ist, dass die *Gestalt* die Abfrage bestimmt. Blaze-Persistence-Entity-Views sind Interfaces aus Gettern über einer Entität (`@EntityView(Cat.class) interface CatNameView { @IdMapping Long getId(); @Mapping("name") String getCatName(); }`), mit verschachtelten Subviews, Collection-Mappings, `@Limit(limit="5", order={"age DESC"})` pro Collection, Attributfiltern und Keyset-Paginierung, und die Engine leitet SELECT und Joins ab; ihre GraphQL-Integration lässt Joins für nicht angeforderte Felder weg (Blaze manual). Die geschlossenen Interface-Projektionen von Spring Data lassen die Engine „die Abfrageausführung optimieren, weil wir alle Attribute kennen“ (Spring Data projections). jOOQs `MULTISET` sammelt eine Kind-Unterabfrage in einer Anweisung in eine verschachtelte Collection, nativ auf Oracle und über JSON emuliert auf MariaDB/MySQL, und implizite Pfad-Joins (`BOOK.author().FIRST_NAME`) leiten den JOIN aus dem FK-Graphen ab, „ohne Performance-Nachteil“ (jOOQ MULTISET, jOOQ implicit join). Die ERPs haben das deklarative Lesemodell als Daten: OFBiz-View-Entities mit Mitgliedsentitäten, `view-link` (Outer Join über `rel-optional`), Aliasse mit `group-by` und `function="sum|count|..."` (OFBiz Entity Engine Guide); Query-Objekte in Business Central mit `DataItemLink`, `SqlJoinType` und `Method = Sum|Count|...`, die implizit nach den übrigen Spalten gruppieren (BC Query object, aggregation); iDempiere-Info-Windows mit FROM-Klausel, Kriterienspalten mit Operator und SQL-Funktion, Identifikatorspalten, Paging und rollenbasiertem Einfügen von Zugriffsfiltern (iDempiere Info Window). OutSystems-Aggregates holen nur die tatsächlich verwendeten Attribute und überstehen Umbenennungen, weil Attribute auf das Modell verweisen (OutSystems Aggregate); OpenXavas `@Tab` ist ein benanntes Listen-Lesemodell mit Spalten einschließlich Pfaden, einer Basisbedingung und einer Standardsortierung (OpenXava tab). Ergebnisfenster gehören zur selben Idee: Spring Data trennt Page (Count-Abfrage), Slice (Größe plus eine Probe) und Keyset-Window, das „die Kriterienklausel so umschreibt, dass alle Sortierfelder und der Primärschlüssel enthalten sind“, und nicht-null Sortierschlüssel braucht (Spring scrolling, query methods); jOOQs SEEK emuliert Row-Value-Prädikate je Dialekt und lässt sich nicht mit OFFSET kombinieren (jOOQ SEEK).

Für ManMap ist das ein Konzept `view <Name> on <EntityMapping>` innerhalb einer Persistence Description, dessen Mitglieder Punktpfade über gemappte Felder und Referenzen sind (`kunde.name`), verschachtelte Listen-Views mit optionalem Limit pro Elternteil und Aggregatmitglieder mit `group`

`by`. Für eine `READONLY`-Repository-Methode `load view RechnungInfo where ... window (page | slice | keyset)` werden SELECT, Joins (INNER für nicht nullbare Referenzen, sonst LEFT), das Holen verschachtelter Collections (MULTISET auf Oracle, JSON-Aggregation oder eine zweite IN-Listen-Abfrage auf MariaDB/MySQL) sowie das Count- oder Seek-Prädikat generiert. MPS kann Projektion, Join-Pfade und Bind-Stelligkeit im Editor prüfen; das ist der strukturelle Gewinn, den jOOQ gegenüber Strings bietet. Der Zieltyp ist ein DTO, sodass das Lesemodell wie heute außerhalb der Identity Map bleibt.

In MoWare heute. Ein `where`-Filter kann nicht aggregieren, projizieren oder über deklarierte `refJoin`/`listJoin` hinaus joinen. Daher ist jede Suche und jede Listenansicht ein `<Thema>LeseRepo` mit handgeschriebener `SQL query`, dynamischen `if`-Fragmenten pro Kriterium und einer `nokeystore/read-only map` oder einem `row mapper` in ein DTO, dessen Property-Liste von Hand synchron gehalten werden muss (MM § „Custom SQL mit `SQL`“; § „Row-Mapper und No-Key-Mapper“; CONV `mw-anwendungsaufbau_v1.md` § „`domain`, `read` und `unit`“). SQL-Text ist im Modell „Wort für Wort ein eigener Knoten“, und das Binden einer Status-Variable war so unklar, dass eine Suche als gemapptes `where` neu gebaut wurde (PR2 P-033). Paging ist nur `limit`, ohne Count- oder Keyset-Unterstützung (MM § „Gemappte Abfragen mit `QueryFromMap`“).

C3. Deklarierte Aggregat- und abgeleitete Properties

FlowFields in Business Central „sind keine physischen Felder ... Sie sind die Beschreibung einer Berechnung“: `Sum("Customer Entries".Amount where(CustNo=field(CustNo)))`, `Count`, `Exist`, `Min`, `Max`, `Lookup`, bei Bedarf per SQL berechnet, parametrisiert über vom Benutzer setzbare FlowFilter und beschleunigt durch indizierte SIFT-Views oder Columnstore-Indizes. Die dokumentierten Fallstricke sind, dass ausgeblendete FlowFields bis v26 trotzdem berechnet wurden, dass im Code gelesene FlowFields „immer null sind“, sofern nicht `CalcFields` aufgerufen wird, und dass die SIFT-Pflege Schreibaufwand und Sperrprobleme mit sich bringt (BC FlowFields, FlowFilters, SIFT and SQL Server, SIFT to NCCI). Berechnete Felder in Odoo deklarieren `@api.depends('value','tax')`, werden standardmäßig nicht gespeichert und sind mit `store=True` durchsuchbar und gruppierbar; ein Blog eines einzelnen Autors dokumentiert Neuberechnungsstürme durch zu weit gefasste Abhängigkeiten (Odoo ORM, dev.to). OpenXava trennt drei Varianten: `@Calculation` als Ausdruck über Properties derselben Entität, live neu berechnet, eine berechnete Property nur mit Getter und `@Depends`, und `@Formula` als SQL-Fragment, das die Datenbank berechnet und das in Listen filterbar ist (OpenXava model); Axelor hat Formel-Felder (SQL) gegenüber berechneten Feldern (Java) (Axelor models); iDempiere hat virtuelle Spalten über `Column SQL` (ADempiere AD overview); Hibernates `@Subselect`/`@Immutable` bildet eine schreibgeschützte Entität auf ein SQL-Select ab (Hibernate @Subselect).

ObjectFlow hat bereits virtuelle Business Properties; die Ergänzung ist eine Art `aggregate`, deren Rumpf ein deklariertes Aggregat über ein anderes EntityMapping ist (`sum(Zahlung.betrag) where Zahlung.rechnung == this`). Es wird von einer generierten SQL-Anweisung nur dann berechnet, wenn eine Repository-Methode es ausdrücklich anfordert (`load ... with aggregates(offenerBetrag)`), nie implizit, optional parametrisiert über Filter des Aufrufers, mit einer Hinweisoption `INDEX`. Eine Liste `depends on` für im Speicher abgeleitete Properties erlaubt dem Generator, ungültig zu machen oder neu zu berechnen, und dem Checker, Selbstabhängigkeit zurückzuweisen. Die Lehre aus BC, dass gerade die implizite Berechnung beißt, passt zum expliziten Stil von ManMap: Die DSL sollte die Berechnung an der Aufrufstelle sichtbar machen.

In MoWare heute. Das `get` einer virtuellen Property läuft bei jedem Lesen und „können keine Repositories oder Services aufrufen“. Jedes Aggregat über Kindzeilen wird daher entweder als vollständige Liste geladen und im Speicher summiert oder über eine handgeschriebene `SQL query` in

ein DTO geholt, und in Listen angezeigte Summen sind Spalten eines `<Thema>LeseRepo`-Lesemodells (OF § „Virtuelle Business Properties“; MM § „Lesemodell mit Custom SQL verwenden“).

C4. Benannte Load-Graphen pro Anwendungsfall

Jmix-Fetch-Plans deklarieren, „welcher Objektgraph … in einer bestimmten Operation eager geladen werden soll“, sind benannt und wiederverwendbar (`_local`, `_base`, eigene Pläne in `fetch-plans.xml`, `extends`) und lassen ausgelassene Attribute leer, sodass ein Zugriff „Cannot get unfetched attribute“ wirft (Jmix fetching). Hibernates Leitfaden nennt JPA-Entity-Graphs „nicht typsicher und unnötig wortreich“, schätzt aber die Idee, und bietet Fetch-Profile an, die gezielt Subselect-Fetching und Batch-Fetching über IN-Listen anfordern, als einzige Alternative zu Joins, die keine Lazy-Proxies braucht (Hibernate Introduction). Die Kritik an Lazy Loading ist eindeutig: N+1 entgeht Slow-Query-Logs, weil jede einzelne Abfrage schnell ist, und Open Session in View hält die Verbindung während des Renderns, daher „zwingt eine DTO-Projektion den Anwendungsentwickler, genau die benötigte Datenmenge zu holen“ (Vlad Mihalcea N+1, OSIV). SOQL drückt dasselbe pro Abfrage mit Eltern-Punktpfaden und Kind-Unterabfragen aus (Trailhead SOQL).

ManMaps „nicht geladene Referenz wirft, nicht geladene Liste ist leer“ ist bereits Jmix' Vertrag für Teilentitäten. Eine Deklaration `load graph <Name> on <EntityMapping> { kunde; positionen { artikel } }`, angehängt an `CHECKOUT`/`READONLY`-Methoden, mit einer Strategie pro Kante (Join, IN-Listen-Batch, separate Abfrage), verträgt sich daher mit der Haltung „kein Lazy Loading“; derselbe Graphname kann von DataUX-Bindings und von einem API-Vertrag referenziert werden. Ein Checker kann warnen, wenn der Rumpf einer Methode eine Referenz berührt, die nicht in ihrem deklarierten Graphen ist.

In MoWare heute. Jede `checkout(id)`-Methode lädt die Wurzel mit `get`, weist dann Referenzen und Listen über separate Abfragen zu oder deklariert `refJoin`/`listJoin` pro Abfrage, und „Eine leere Liste beweist daher nicht, dass in der Datenbank keine Kindzeilen vorhanden sind“. Ist eine Referenz nicht geladen, liest der Code `#Key` und ruft `# Repo.get(...)` von Hand auf (MM § „Explizites Laden“; OF § „Beziehungen und Objektgraphen“). Der geladene Graph steckt implizit im Methodenrumpf und lässt sich nicht über einen Namen wiederverwenden.

C5. Implizite Zeileneinschränkungen: Soft Delete, Mandantenfähigkeit, benannte Filter und Row-Level Security

Ein Mechanismus deckt vier Bedürfnisse ab. Hibernate 6.4 machte Soft Delete deklarativ (`@SoftDelete` mit Strategie ACTIVE/DELETED, Kennzeichenspalte und Konverter; ein Delete wird zu einem UPDATE und Entitätsabfragen blenden die Zeilen aus, natives SQL nicht), machte Mandantenfähigkeit über Diskriminator zu einer `@TenantId`-Spalte mit automatischer Filterung („native SQL-Abfragen werden nicht automatisch nach Mandanten-ID gefiltert“) und bietet benannte, parametrisierte Einschränkungen `@FilterDef`/`@Filter`, die pro Session aktiviert und bewusst nicht auf `find()` oder `@ManyToOne` angewendet werden (Hibernate User Guide §3.12, Hibernate Introduction). Jmix dokumentiert die scharfen Kanten von Soft Delete: Eindeutigkeits-Constraints brauchen datenbankspezifische Tricks, und To-one-Referenzen können ins Leere auf gelöschte Zeilen zeigen (Jmix soft deletion). Row-Level Security ist ein deklariertes Prädikat pro Rolle in Jmix (`@JpqlRowLevelPolicy(where="c.active = TRUE")` mit `{E}`-Alias und Session-Parametern, nur für die Wurzelentität), Odoo (`ir.rule`-Domains, globale Regeln mit AND, Gruppenregeln mit OR verknüpft), iDempiere (`MRole.addAccessSQL` hängt Mandanten-/Organisations- und Zugriffsklauseln an jede Info-Window- und Report-Abfrage an), Mendix (XPath-Constraints an Zugriffsregeln, beim Abruf angewendet und vor dem Client verborgen) und Business Central (Sicherheitsfilter an Berechtigungssätzen plus ein `SecurityFiltering`-Modus pro Datensatz Filtered/Validated/Ignored, wobei Validated langsam ist und Ignored stillschweigend umgeht) (Jmix row-level roles, Odoo

security, MRole.java, Mendix access rules, BC security filters). Causeways SecMan leitet seinen Berechtigungsbaum aus dem Metamodell ab, räumt aber ein, dass seine Mandantenfähigkeit „nur oberflächlich“ ist (Causeway SecMan).

Für ManMap würde ein Konzept `restriction <Name>(params) on <EntityMapping> where <gemapptes Prädikat>`, mit `SOFT_DELETE(column, strategy)` und `TENANT(column)` als vordefinierten Einschränkungen und rollengebundenen Einschränkungen, die im Root `Roles and Permissions` deklariert werden, per AND in jedes generierte `get`/`where`/`reload` eingefügt und die Spalte beim INSERT setzen. Dazu käme ein Modus pro Methode (`filtered`, `unrestricted`) und die dokumentierte Regel, dass eigene `SQL`-Blöcke in der Verantwortung des Entwicklers liegen. Soft Delete macht `DELETE`-Methoden zu UPDATEs und ergänzt einen Abfragemodifikator `deleted also` für die Administration.

In MoWare heute. Sichtbarkeit ist eine Scope-Funktion, die die zugänglichen Objekte eines Typs liefert, über `setScope` in die Page-Metadaten gebunden und im Command ausgewertet wird, nicht im Repository. Jede Repository-Abfrage, die sie beachten muss, wiederholt daher das Prädikat; `CAN_OPEN_RO`/`CAN_OPEN_RW` schützen Commands, nicht Zeilen (OF § „Rollen, Scopes und Identities“; § „Scopes“). Es gibt kein Soft Delete und keine Konvention für eine Mandantenspalte; ein Kennzeichen nach Art von `IsActive` ist eine gewöhnliche Status-Property, an deren Prüfung jedes `where` denken muss.

C6. Historisierung und fachliche Änderungsverfolgung

Der Weg über die Datenbank ist am günstigsten, wo verfügbar: MariaDB `WITH SYSTEM VERSIONING` erzeugt bei jedem UPDATE einer versionierten Spalte eine Historienzeile, beantwortet `FOR SYSTEM_TIME AS OF`, `BETWEEN` und `ALL`, partitioniert die Historie mit `PARTITION BY SYSTEM_TIME` und fügt `row_end` dem Primärschlüssel hinzu, mit Einschränkungen bei Replikation und Dump (MariaDB system-versioned tables); Oracle bietet Flashback Query `AS OF TIMESTAMP`, Version Query mit `VERSIONS_*`-Pseudospalten und Flashback-Time-Travel-Archive, die das Hinzufügen von Fremdschlüssel-Constraints verbieten (ORA-55610) (Oracle Flashback); MySQL hat keines von beiden. Der portable Weg über die Anwendung ist Envers: `@Audited` schreibt pro Änderung eine `_AUD`-Zeile mit `REV` und `REVTYPE`, die `ValidityAuditStrategy` ergänzt `REVEND`, sodass Lesezugriffe auf einen Zeitpunkt zu einer Bereichsbedingung werden statt einer korrelierten `MAX(REV)`-Unterabfrage, und `AuditReader.find(Class, id, Date)` liefert den Zustand zu einem Zeitpunkt (Vlad Mihalcea Envers, AuditReader). Fowler rät von bitemporaler Modellierung ab, sofern rückwirkende Änderungen keine echte Anforderung sind (Fowler bitemporal). Fachliche Änderungsverfolgung ist ein anderes Bedürfnis: Axelors `<track><field name="confirmDate" on="UPDATE" if="status == 'confirmed'"/><message if="status == 'confirmed'" on="UPDATE">Order confirmed.</message></track>` erzeugt bedingte, menschenlesbare Meldungen, die am Datensatz aufgereiht werden, asynchron nach der Transaktion (Axelor messaging); Odoos `tracking=True` pro Feld schreibt `mail.tracking.value`-Zeilen (Odoo mixins); iDempieres „Maintain Change Log“ und das Change Log von Business Central schreiben alte und neue Werte pro Spalte, letzteres mit der Warnung, Postentabellen wegen des Volumens nicht zu protokollieren (PO.java, BC change log); Causeway koppelt ein Command-Log (Absicht) mit einem Audit-Trail auf Property-Ebene (Wirkung), korreliert über eine Interaktions-ID (Causeway audit trail, command log); das Entity Log von Jmix wird pro Entität und Attribut konfiguriert, ist aber blind für natives SQL (Jmix entity log).

Für ManMap ist eine Tabellenoption `AUDITED` die portable Wahl über Oracle, MySQL und MariaDB hinweg. Sie generiert die `_AUD`-Tabelle im Gültigkeitslayout und REVINFO (unter Wiederverwendung der Mechanik von `ADDITIONAL_TABLE_NAME`), das zusätzliche INSERT in `CHECKIN`/`DELETE` und einen Modifikator `as of <timestamp>` an `READONLY`-Methoden. Eine Option `SYSTEM_VERSIONED` kann die MariaDB-Klausel (und auf Oracle das Flashback Archive, mit dem FK-Vorbehalt) erzeugen, wo die Datenbank bekannt ist. Für ObjectFlow ist ein `track`-Block an einer Entity mit `on create|update if`

`<Bedingung>` pro Property und Meldungsvorlagen, beim Check-in geschrieben, das fachliche Gegenstück. Beides mit dem Namen des festschreibenden Commands und dem Benutzer der Session zu korrelieren, liefert Causeways Kopplung von Absicht und Wirkung umsonst, weil die Session beides bereits kennt.

In MoWare heute. ManMap bietet `CREATEDAT`/`CREATEDBY`/`MODIFIEDAT`/`MODIFIEDBY` und `OPTIMISTIC_LOCK` (dessen erforderliche zusätzliche `TCN`-Spalte undokumentiert war, PR2 P-013) und sonst nichts: keine Historientabelle, keine As-of-Abfrage, kein Änderungsprotokoll (MM § „Optimistic Locking und Audit“). Anwendungen, die „wer hat was geändert“ brauchen, schreiben ihre eigene Log-Entity und fügen aus jeder Übergangs-Service-Method darin ein.

C7. Modellgetriebene Schemaevolution und geprüfte Abkündigung

Beide gängigen Werkzeuge versionieren das Schema als geordnete Liste unveränderlicher Änderungseinheiten in einer Historientabelle: Liquibase-Changesets (ID, Autor, Pfad) mit Preconditions, Rollback und `DATABASECHANGELOG`, dazu ein `diff-changelog`, das zwei Schemata vergleicht, oder über die Erweiterung `liquibase-hibernate` ein Hibernate-Modell mit einer Datenbank; das eigene Wiki zeigt, dass eine umbenannte Klasse eine neue Tabelle statt einer Umbenennung ergibt (Liquibase changelogs, diff-changelog, liquibase-hibernate wiki). Flyway kennt versionierte, wiederholbare und Undo-Migrationen mit prüfsummengesicherten, unveränderlichen Skripten (Flyway migrations; Namenskonventionen aus Sekundärquellen). Hibernates `hbm2ddl update` „ist zu vermeiden“ in Produktion (Vlad Mihalcea hbm2ddl). Business Central zeigt Evolution ohne Migrations-DSL: `ObsoleteState` Pending erzeugt Compiler-Warnungen und Removed Fehler bei Referenzen, Upgrade-Codeunits mit Upgrade-Tags behandeln Daten, und Schemaänderungen sind online nur additiv („Löschen Sie keine Primärschlüssel … Fügen Sie keine weiteren eindeutigen Schlüssel hinzu“) (BC ObsoleteState, upgrading extensions, table keys); metasfresh zeichnet jede Änderung am Application Dictionary als SQL-Migrationsskript mit Zeitstempel auf (log_migration_script.md).

Der strukturelle Vorteil einer DSL ist, dass die Modellhistorie die *Absicht* kennt: Eine Umbenennung, eine Typerweiterung oder die Aufteilung eines eingebetteten Value Objects sind in einem Modell-Diff sichtbar, in einem Datenbank-Diff aber nicht. Der realistische ManMap-Entwurf ist ein Generator, der aus dem Diff zwischen zwei Modellversionen (oder einem Modell und einem Datenbank-Snapshot) pro Dialekt ein zu prüfendes Liquibase-XML- oder Flyway-SQL-Changeset erzeugt, das über die übliche Historientabelle verfolgt und nie automatisch angewendet wird. Dazu kommt eine Option `DEPRECATED` an Mappings und Properties mit den Zuständen Pending/Removed, die der Checker wie bei BC zu Warnungen und Fehlern macht.

In MoWare heute. „ManMap ist kein allgemeines Werkzeug zur Migration bestehender Datenbankschemata“. Der Entwickler erzeugt das Schema in MPS aus der generierten Beschreibung, Schemahinweise sind ausdrücklich „keine Schema-Migration“, die generierte Beschreibung verwendet Oracle-Syntax (PR2 P-023), `INDEX` erzeugte stillschweigend einen eindeutigen Index (PR2 P-022), und jede spätere Änderung ist ein handgeschriebenes ALTER (MM § „Datenbankportabilität und Schema“; WB § „Von der Modellierung zur Ausführung“).

C8. Mengenbasierte Schreibzugriffe und dialektabhängig emuliertes Upsert

JDBC-Batching „hilft nur in Fällen, in denen ein Programm viele Inserts, Updates oder Deletes“ gegen eine Tabelle ausführt, und mengenbasierte HQL-`update`/`delete` sind „noch besser“ (Hibernate Introduction); Spring Datas `UpdateSpecification`/`DeleteSpecification` verwenden Abfrageprädikate für Massenanweisungen wieder (Spring Data Specifications); jOOQ rendert eine einzige Upsert-Deklaration als `ON DUPLICATE KEY UPDATE` auf MySQL/MariaDB und als `MERGE INTO ... USING` auf Oracle

(jOOQ upsert); MyBatis Dynamic SQL weigert sich, eine Anweisung zu rendern, deren optionale WHERE-Bedingungen alle weggefallen sind, weil sie „am Ende alle Zeilen betreffen könnte“ (MyBatis Dynamic SQL).

Für ManMap ist das eine Art von Repository-Methode, gemapptes `update <EntityMapping> set <field> = <expr> where <gemapptes Prädikat>` und `delete <EntityMapping> where ...`, mit dem Schutz gegen leeres WHERE, sowie eine explizite Speichervariante `UPSERT` pro EntityMapping, die der Generator je Dialekt emuliert. Weil Upsert die TCN-Prüfung umgeht, muss es ein eigener Methodentyp sein, keine Option an `CHECKIN`.

In MoWare heute. Mengenbasierte Änderungen sind ein `STATEMENT`-SQL-Block mit handgeschriebenem, dialektgebundenem Text; `BATCH` existiert, ist aber auf MySQL/MariaDB nicht verfügbar und scheitert dort mit Auto-IDs (MM § „`SQL query` und `SQL statement`“; § „Save-Optionen“; PR2 P-040).

Thema D. Verträge, Probleme und Module

D1. Deklarierte Probleme an Service Methods und ein abgeleiteter API-Vertrag

Rusts Position ist, dass `Result` der Standard ist, „wenn Sie eine Funktion definieren, die fehlschlagen könnte“, und `panic!` für Vertragsverletzungen gedacht ist, die auf einen Fehler beim Aufrufer hindeuten, wobei `?` Fehler weiterreicht und konvertiert (Rust book ch09-02, ch09-03); Kotlin hat Checked Exceptions abgeschafft und empfiehlt für erwartete Abwesenheit Varianten, die null zurückgeben (Kotlin exceptions); GraphQL liefert `data` plus eine `errors`-Liste mit `message`, `path` und `extensions` und reicht den Fehler eines nicht-null Feldes an den nächsten nullbaren Elternteil weiter (GraphQL spec); OpenAPI deklariert Antwortschemata pro Statuscode mit den Bereichen `4XX`/`5XX` und einem `default` (OAS latest). Fehlersammlung ist das, was UIs brauchen: Irons `refineValidation` sammelt, Mendix zeichnet alle Meldungen fehlschlagender Regeln auf, und `[ErrorBehavior(ErrorBehavior::Collect)]` in Business Central sammelt, statt anzuhalten (Iron, Mendix validation rules, BC collecting errors). Mendix' `Validation Feedback(object, member, message)` und die feldgebundene Validierungsmeldung von Salesforce zeigen, dass ein Problem die betroffene Property benennen sollte (Mendix validation feedback, Trailhead validation rules). Mendix leitet aus Entitäten und Microflows einen OData-v4-Service mit einem OpenAPI-3.0-Dokument und optionalem GraphQL-Schema ab; das generische REST von Jmix liest dieselbe Sicherheit wie die UI; Causeway implementiert Restful Objects über dem Metamodell (Mendix published OData, Jmix REST, Causeway RO viewer).

Der ObjectFlow-Teil ist eine Klausel `problems { NichtGenugBestand, KundeGesperrt }` an Service Methods, sodass die erwarteten Fehlschläge Teil der Signatur werden und generierter Code ein versiegeltes Ergebnis liefert; dazu eine Form `problem(property, message)` der Precondition, die die Meldung an eine Business Property bindet, und ein Sammelmodus für Preconditions bei direkten Service-Aufrufen. Daraus plus A3 (Nullbarkeit), A4 (versiegelte DTO-Alternativen als `oneOf`/Union) und einer Konvention für Page- oder Cursor-DTOs wird ein OpenAPI-3.1-Dokument für jede `API_METHOD` zu einer Generatorausgabe, mit den bestehenden JSON-Serdes als Implementierung. Additive Evolution mit `deprecated`-Flags ist der günstigere Versionierungsweg, wie GraphQLs versionslose Praxis nahelegt (GraphQL schema design). Die API-Ableitung selbst ist nur bis hierhin im Umfang; UI- und Transportschicht sind es nicht.

In MoWare heute. `API_METHOD` „markiert für API-Integration“ und ändert sonst nichts; es gibt kein Vertragsartefakt (OF § „Service-Methoden“). Eine verletzte Precondition bei einem direkten Service-Aufruf wirft `OFXAbortedException` und lässt die Probleme an der Session zurück, sodass spätere Aufrufe in derselben Session ebenfalls fehlschlagen (PR2 P-030). Eine Precondition-Meldung ist ein `StringFormatString` ohne Property-Bezug, und `validation` sammelt Verstöße nur innerhalb eines Command-Ablaufs (OF § „Preconditions, Validation, Guards und Exceptions“).

D2. Modulgrenzen als geprüfte Constraints und ein generierter Modul-Steckbrief

Spring Modulith behandelt jedes Unterpaket als Modul mit einer bereitgestellten Schnittstelle (API-Beans und publizierte Events), Interna, die „von anderen Modulen nicht referenziert werden dürfen“, und deklarierten erlaubten Abhängigkeiten. `ApplicationModules.of(App.class).verify()` schlägt bei einem zyklischen Abhängigkeitsgraphen oder einer internen Referenz fehl, und sein `Documenter` erzeugt C4-Diagramme und einen Steckbrief (Canvas) pro Modul, der Aggregate, publizierte und konsumierte Events sowie Konfiguration auflistet (Modulith fundamentals, verification, documentation). Causeways SecMan leitet aus dem Metamodell einen Berechtigungsbaum über Namensraum, Typ und Member ab, nach dem Prinzip „das Spezifischste gewinnt“ (Causeway SecMan). OutSystems weigert sich, eine Action öffentlich zu machen, wenn ihre Parameter nicht freigegebene Entitäten verwenden (OutSystems Server Action).

In einer projektionalen DSL werden daraus Typsystemfehler statt Tests: Die Importtabelle der Konventionen (`base`, `rollen`, `sharedvalues`, `gate.*`, `<bereich>.domain`, `read`, `unit`, `app`, `tests`), die Regel „keine Zyklen“ zwischen Bounded Contexts und die Regeln zur Platzierung von DTOs lassen sich einmal pro Modell als `layer` deklarieren und zur Bearbeitungszeit prüfen. Ein generierter Steckbrief (Entities, Commands, Services, publizierte und konsumierte Events pro Bereich) folgt direkt aus dem Modell und würde die von Hand gepflegten Übersichtstabellen in den Konventionen ablösen.

In MoWare heute. Die Schichtung ist „strikte Importtabelle; keine Importe höherer Ebenen; keine Zyklen zwischen Bounded Contexts“, nur per Konvention und Review (CONV `mw-anwendungsaufbau_v1.md` § „Solutions und Modelle“, § „Abhängigkeiten“). Ein Verstoß zeigt sich als Entwurfsdiskussion (PR2 P-050), nicht als Checker-Fehler.

Drei Laufzeitfähigkeiten tragen die meisten Ideen

Zusammen gelesen reduzieren sich die Themen auf drei Investitionen in Laufzeitumgebung und Generator, von denen jede mehrere Ideen auf einmal freischaltet. Erstens macht eine *phasierte* Session als Unit of Work mit benannten Einhängepunkten (Determinations, Validations, Save, nach dem Commit) aus A2, B3, C6 und der fachlichen Änderungsverfolgung in C6 generierten Code beim Check-in, weil die Session bereits die Transaktion besitzt und den Benutzer und das Command kennt; RAPs Strict Mode und die feste Pipeline von Salesforce zeigen beide, dass erst das Benennen der Phasen Regeln kombinierbar und prüfbar macht. Zweitens ist ein *persistierter Scheduler mit Arbeitswarteschlange* (eine oder zwei Tabellen im Stil von db-scheduler und den Workpackages von metasfresh) das gemeinsame Fundament für B4, B5, B6 und das Leeren der Registry in B3; ohne ihn ist jedes „später“ in einer MoWare-Anwendung eine Producer-Abfrage, die Arbeit neu findet. Drittens macht ein *graphbewusster ManMap-Generator*, der den deklarierten FK-Graphen liest, C1, C2, C3, C4 und C8 mechanisch, und das referenzbasierte Bearbeiten in MPS liefert bereits die Umbenennungssicherheit, mit der OutSystems und jOOQ werben.

Ebenso wichtig ist, was die Belege *nicht* zu kopieren raten. Lazy Loading (N+1, OSIV), implizite Kaskade (Vlad Mihalceas Kritik, Axelors Voreinstellung `orphanRemoval`), implizite Aggregatberechnung (BCs ausgeblendete FlowFields), stillschweigend überspringbare Regeln (BCs `RunTrigger`), Regel-Engines mit Agenden (Drools, Easy Rules) und Event Sourcing als Quelle der Wahrheit (Dudycz, Fares) sind allesamt dokumentierte Schmerzpunkte; die MoWare-Prinzipien explizites Laden, explizites Speichern und Prüfen vor dem Ändern werden bestätigt. Die obigen Ideen sind daher als *deklarierte, sichtbare, nicht transitive* Varianten formuliert: eine Komposition, die genau die Anweisungen „erst Eltern, dann Kinder" generiert, die man sonst selbst geschrieben hätte, eine View, deren SELECT man lesen kann, ein Aggregat, das nur dort berechnet wird, wo es angefordert ist, Regeln, die immer beim Check-in laufen, und Events als Benachrichtigungen, während der Zustand die Wahrheit bleibt.

Vermutete wiederkehrende Handarbeit

Abschnitt 5 der Baseline-Notiz listet die Strukturen, die jede Anwendung pro Aggregat, pro Suche oder pro Testmodell neu implementiert, und Abschnitt 6 die Sprachgrenzen, auf die Anwender in den beiden Problemprotokollen tatsächlich gestoßen sind. Die Namensempfehlungen der Konventionen (`<Aggregat>Repo`, `<Aggregat>Service`, `<Bereich>Persistenz`, `<Bereich>Lesen`, `<Thema>LeseRepo`, `<Bereich>Tests`) sind selbst ein Beleg dafür, dass diese Artefakte identisch wiederkehren. Die Tabelle ordnet jedes Muster der Idee oder den Ideen zu, die es aufnehmen würden. Wo ein Punkt nur teilweise aufgenommen wird, ist der Rest vermerkt.

Handgeschriebenes Muster (was jedes Mal geschrieben wird)	**Wo dokumentiert**	**Aufgenommen durch**
Suchfilter-DTO mit `results: list<Info-DTO>`, Konstruktor setzt Status-Properties auf `null` zurück	WB § „Gesamtbeispiel", § „Rechnungen suchen"; PR2 P-034	C2 (View mit Filterparametern und Ergebnisfenster), A3 (nullbarer Status braucht kein Zurücksetzen im Konstruktor)
Such-Command mit Filterseite und Ergebnisseite, `page <same page>` als Aktualisierung	WB § „Rechnungen suchen"; OF § „Page Conclusions"	C2 (generierte View-Abfrage); Aktualisierung nach Bearbeitung über B3 (Event-Handler führt die View erneut aus) statt `any cmd term` (PR2 P-044)
`<Aggregat>Repo` mit `checkout(id)`, das Wurzel plus Listen/Referenzen lädt, und `checkin(entity)`	WB § „Persistenz mit ManMap"; CONV `mw-anwendungsaufbau_v1.md`	C1 (`save graph with`), C4 (benannter Load-Graph an `CHECKOUT`)
Speicherreihenfolge erst Eltern, dann Kinder, Weitergabe von Schlüssel oder Rückreferenz	MM § „Speichern von Objektgraphen"	C1
Löschen der Kinder vor dem Elternteil	MM § „Löschen mit `delete with`"	C1 (`delete graph with`), mit `DENY`/`UNLINK` für Referenzen von außen
Erkennen entfernter Kinder durch Merken der geladenen Liste und Abgleich in `FINAL OK_CONCLUSION`	PR2 P-041	C1 (Löschen verwaister Kinder per Schlüsselabgleich)
`<Thema>LeseRepo` mit `SQL query` und `nokeystore/read-only map` in ein DTO; eigene Persistence Description `<Bereich>Lesen`	WB § „Persistenz mit ManMap"; MM § „Lesemodell mit Custom SQL verwenden"	C2; Aggregatspalten durch C3

Handgeschriebenes Muster (was jedes Mal geschrieben wird)	**Wo dokumentiert**	**Aufgenommen durch**
Dynamisches SQL über `if` pro Kriterium	MM § „Custom SQL mit `SQL`“	C2 (optionale Prädikate in Views) und die Regel für optionale Where-Terme unter Kleinere Verbesserungen
Platzhalter `%` für vollständig optionale `||`-Gruppen	MM § „Spezifikum - Filterausdrücke“; PR2 P-037	Kleinere Verbesserungen (Semantik optionaler Terme mit neutraler `||`-Gruppe)
Explizites Laden von Referenzen und Listen, `#Key` plus `# Repo.get(...)`, wenn nicht geladen	MM § „Explizites Laden“; OF § „Beziehungen und Objektgraphen“	C4
Service Method für Zustandsübergang mit Status-Preconditions, plus reine Variante `...Entscheiden(x, fakten)`	CONV `mw-domaenenmodell_v1.md` § „Aggregate und Invarianten“, § „Service und Aggregat“	B1 (generiertes Gerüst und Precondition auf den Quellstatus), A2 (Validations beim Speichern), B2 für Belegaggregate
Fakten-DTO für aggregatübergreifende Regeln, vom Aufrufer geladen und an den Service übergeben	CONV `mw-domaenenmodell_v1.md`; PR2 P-050	A6 (Spezifikation über mehrere Aggregate, zu einer Abfrage kompiliert), B3 (Eventual Consistency über Events statt synchroner Fakten)
Block „erst validieren, dann mutieren“	OF § „Preconditions, Validation, Guards und Exceptions“	A2 (Phasenreihenfolge durch die Session erzwungen)
Bearbeitungs-DTO zwischen UI und Aggregat, Werte einzeln an den Service übergeben, Eingabeprüfungen über `LENGTH`/`RANGE`	WB § „Rechnung und Positionen bearbeiten“	A1 (Constraints einmal deklariert, an der Grenze DTO zu Entity geparst), D1 (an Properties gebundene Probleme)
Paar aus Besitzer- und Bearbeitungs-Command mit `DISABLED`-Formular und Revert der gemeinsamen Wurzel	WB § „Rechnung und Positionen bearbeiten“; CONV `mw-commands_v1.md`	Nicht aufgenommen (UI-seitiges Muster, außerhalb des Umfangs); B7 würde die Persistenz der gemeinsamen Session ergänzen
Termination-Handler, der das gepushte Objekt in `filter.results` einfügt und Werte von Hand in DTOs kopiert	OF § „`cmdTermHandler`“; CONV `mw-commands_v1.md` § „Suche und Rückgabe“	C2 (View für den geänderten Schlüssel erneut ausführen) oder B3 (Handler auf das Event der Entity aktualisiert die Ergebnisliste)
Scope-Funktion, die erlaubte Werte in `command init` lädt und `setScope`/`setElements`/`setEnabled` bindet	OF § „Scopes“	B1 (freigegebene Übergänge aus der Übergangstabelle abgeleitet), C5 (Zeileneinschränkungen im Repository ausgewertet)
Vermeiden doppelter Checkouts über `session entities (keys of checked out)` oder `#+ with`	OF § „Entities in der Session prüfen“	C4 (ein einmal pro `CHECKOUT`-Methode deklarierter Load-Graph vermeidet überlappende Ladevorgänge); bleibt teilweise bestehen
Rumpf von `isAuthenticated`, der Benutzername und ID setzt; auch am `BatchJob Module` Pflicht, aber ungenutzt	OF § „User Environment und User Service“; DUX § „Batchjob“	Kleinere Verbesserungen (Deklaration eines Authentifizierungsproviders in `OFXConfig`); keine Konzeptfamilie der Sprache

Handgeschriebenes Muster (was jedes Mal geschrieben wird)	**Wo dokumentiert**	**Aufgenommen durch**
Job-Benutzerkontext in `OFXConfig` (`UserEnvironmentInformation`, `OFXSimpleUserServices`)	OF § „User Environment und User Service“	B6 (persistierte Arbeitselemente tragen Benutzer- und Mandantenkontext vom Zeitpunkt des Einreihens, wie die Workpackages von metasfresh)
`TestDaten`-Service mit `CS`-Custom-Session, `session operation add`, `#CS.COMMIT()`, Zurücklesen in frischer Session	CONV `mw-tests_v1.md`	C1 (Graph-Speichern in einem Aufruf entfernt die meisten `session operation add`-Zeilen); der Namenskonflikt `__testsTestDaten` (PR2 P-055) ist ein Generator-Fix
Stammdaten holen oder anlegen in Tests	CONV `mw-tests_v1.md` § „Plausible Testdaten“	C8 (Speichervariante `UPSERT`)
Test-Repository über `superclass` plus `primary instance`	OF § „Repositories im Test ersetzen“	Nicht aufgenommen; bleibt der Mechanismus zum Ersetzen externer Systeme
Idempotente Consumer-Precondition, die erneut prüft, ob die Arbeit noch nötig ist	DUX § „Batchjob mit `BatchJob Module`“	B6 (persistierte Inbox mit Elementzustand, Schlüssel über `idempotent by`)
Feature-Flags als `static role`	OF § „Rollen, Scopes und Identities“	Nicht aufgenommen; eine kleine Option `feature flag` würde die Praxis nur umbenennen
Schreibgeschützte Benutzerverwaltungs-Entity und Repository in `rollen`	CONV `mw-anwendungsaufbau_v1.md`	Nicht aufgenommen
Statuswechsel-Maske, die das Command zwingt, den alten Status um die Service-Prüfung herum zu merken und wiederherzustellen	PR2 P-042	B1 (Übergang am Status deklariert; die UI bietet Übergänge an, keine rohen Statuswerte)
Grenzen aus dem Command- und Session-Modell laut Problemprotokollen: `revert` nur für Parameter (PR1 P-014), keine Abbruch-Conclusion (PR1 P-017), `<user_cancel>` nur für `SEARCH_CMD` (PR1 P-024), `GRAPH_EDIT_CMD` in Tests nicht auf oberster Ebene (PR1 P-026), seitenlose Bearbeitung liefert einen Wert nur per Push (PR2 P-041)	PR1, PR2	Teilweise B7 (persistierte Session) und B4 (Prozessschritte mit typisierten Daten); überwiegend Generator- und Test-DSL-Fixes statt neuer Konzeptfamilien
Status- und Null-Semantik: genau ein `ON_CREATION`-Element (PR2 P-011), Filter-Status beginnt bei `ON_CREATION` (PR2 P-034), `LocalDate ==` als Referenzvergleich (PR1 P-023), `OPTIONAL`-String-Delegate liefert `null` (PR1 P-032)	PR1, PR2	A3 (deklarierte Nullbarkeit pro Property steuert Voreinstellungen, Operatoren und `status switch`-Zweige)
Lücken bei MySQL/MariaDB: `AUTOID`-Sequenznamen ignoriert (PR1 P-016), `BATCH` mit Auto-ID nicht implementiert (PR2 P-040)	PR1, PR2	C8 (dialektabhängig emulierte Anweisungen) und die Option für formatierte Sequenzen unter Kleinere Verbesserungen
`CAN_OPEN_RO` sperrt alle Formulare, sodass ein gefiltertes `SEARCH_CMD` unbrauchbar ist (PR2 P-060)	PR2	C5 (Zeileneinschränkung pro Rolle statt schreibgeschützter Session), Lese-/Schreibdeklaration auf Member-Ebene unter Kleinere Verbesserungen

Kleinere Verbesserungen

Die folgenden Punkte sind Optionen an bestehenden Konzepten, Checker-Regeln und Komfortfunktionen; jeder umfasst ein oder zwei Sätze mit Quelle. Sie sind nach dem MoWare-Konzept gruppiert, an das sie andocken würden.

Business Properties und Typen

- Feldsteuerung pro Operation: RAPs `field ( readonly : update )`, `field ( mandatory : create )` und `field ( suppress )` sind kombinierbare Feldmodi pro Operation; als Property-Optionen würden sie `#Meta setEnabled`-Aufrufe in Scope-Funktionen ersetzen (SAP cheat sheet 36). Zu beachten ist, dass RAP für `mandatory` außer `mandatory:create` keine Laufzeitprüfung durchführt, daher sollte ObjectFlow beim Check-in prüfen (SAP Learning static field control).
- `status switch` ohne `default` generieren und `case null` nur ergänzen, wenn die geprüfte Property als nullbar deklariert ist, sodass javac ein neues Status-Element in jedem unveränderten Switch meldet und eine `MatchException` die Laufzeit absichert (JEP 441).
- Value Objects und DTOs als Java-Records generieren, mit der Constraint-Prüfung im kompakten Konstruktor; Records können keine Klassen erweitern, daher brauchen DTO-Hierarchien Sealed Interfaces, und der `value record` aus JEP 401 ist später ein Upgrade um ein einziges Token (JEP 395, JEP 401).
- JSpecify-`@NullMarked`/`@Nullable` an generierten Klassen ausgeben, sobald Nullbarkeit deklariert ist (JSpecify user guide).
- Lookup-Vertrag pro Entity: OpenXavas `@SearchKey` (was der Benutzer tippt, um eine Referenz zu finden) und Jmix' `@InstanceName` (menschenlesbare Bezeichnung, mit `@DependsOnProperties`) sind UI-unabhängige Deklarationen, die DataUX und jede API wiederverwenden können (OpenXava model, Jmix entities).
- Erwägenswerte Property-Flags aus Axelor: `translatable`, `encrypted` (AES-256), `massUpdate`, `namecolumn`, `search="a,b"`, `initParam` (Axelor models).
- Drei Varianten abgeleiteter Properties, Ausdruck (live neu berechnet mit `@Depends`), Java-Getter und SQL-Formel, jeweils mit expliziter Depends-on-Liste; OpenXava trennt sie sauber und verbietet `@Depends` an SQL-Formeln (OpenXava model).
- Status-Element an eine Service-Implementierung gebunden (Strategie-Dispatch), wie BCs `enum ... implements Interface` mit `Implementation = I = Codeunit` pro Wert (BC interfaces).
- Money-`allocate` als die eine Operation, die `BigDecimal` fehlt: anteilige Aufteilung ohne Cent-Verlust (Fowler, Money; Details zur Allokation über Verraes, sekundär).
- Checker: Ein Literal, das einer eingeschränkten Property zugewiesen wird, wird im Editor geprüft, so wie Iron `val y: Int :| Greater[0] = -1` zur Compile-Zeit prüft (Iron).

Preconditions, Probleme und Service Methods

- An Properties gebundene Probleme, `precondition ... on rechnung.betrag`, damit UI und API die Meldung platzieren können; Mendix' `Validation Feedback(object, member, message)` und die Fehlerposition bei Salesforce sind die Vorbilder (Mendix validation feedback, Trailhead validation rules).
- Sammelmodus für Preconditions bei direkten Service-Aufrufen, der die vollständige Problemliste zurückgibt, statt beim ersten Problem `OFXAbortedException` zu werfen (PR2 P-030); BCs `ErrorBehavior::Collect` und Irons `refineValidation` sind Präzedenzfälle (BC collecting errors).

- Vorheriger Wert und Änderungsflags in Regeln: Salesforce `ISNEW()`, `ISCHANGED(field)`, `PRIORVALUE(field)`, metasfresh `ifColumnsChanged`, Jmix `AttributeChanges`. Ein Ausdruck `changed(x)`/`old(x)` in Preconditions beschränkt eine Regel auf einen Übergang und vermeidet die Falle, dass bereits bestehende Verstöße unbeteiligte Bearbeitungen blockieren (Trailhead best practices, Jmix entity events).
- Tags für Validierungsphase und Kanal: OpenXavas `onlyOnCreate` und `@RemoveValidator`, Odoos `@api.ondelete` und `Many2one(ondelete=restrict|set null|cascade)` sowie Jmix-Validierungsgruppen (`UiComponentChecks`, `RestApiChecks`) legen nahe, Preconditions mit create/update/delete und der Anwendbarkeit für UI/API/Service zu kennzeichnen (OpenXava model, Odoo ORM, Jmix bean validation).
- Optionen für die Ausführungsstrategie von Services: OFBiz' `use-transaction`, `require-new-transaction`, `transaction-timeout`, `semaphore wait|fail` und `max-retry` benennen, was `#+ with`-Custom-Sessions von Hand tun (OFBiz Service Engine Guide).
- Option für Methodensemantik `SAFE | IDEMPOTENT | NON_IDEMPOTENT`, wie Causeways `@Action(semantics=...)`, wobei sichere Methoden die Nebenläufigkeitsprüfung überspringen und RAP-Functions „keine Daten sperren oder ändern“ (Causeway @Action, SAP cheat sheet 36).
- Eine Teilmenge der Operationen eines Aggregats pro Command oder API freigeben, wie RAP-Projektions-BDEFs mit `use create; use action act;` (SAP cheat sheet 36).
- Checker: Die Parameter- und Rückgabe-DTO-Typen einer `API_METHOD` müssen selbst publiziert sein, die Freigaberegel von OutSystems (OutSystems Server Action).
- Periodensteuerung als Precondition-Familie, gegliedert nach Belegart, Datum, Organisation und Action gegenüber Buchung (MPeriod.java).
- Den Status vor einer Lebenszyklus-Action unter Zeilensperre erneut lesen und mit „Status geändert“ fehlschlagen, wenn eine andere Session ihn verändert hat, wie iDempieres `DocStatusChanged` (DocumentEngine.java); dazu eine Zusicherung `expect status X after` an Übergangsmethoden, wie metasfreshs `processEx` (IDocumentBL.java).
- Semantische Sperre: Ein Status-Element als `pending` markieren, sodass die Preconditions „nicht während pending“ anderer Commands generiert werden (Microsoft saga).
- Entscheidungstabellen-Ausdruck mit Hit-Policy-Enum (Unique, First, Any, Rule order, Collect+SUM/MIN/MAX/COUNT), wobei Unique zur Testzeit geprüft wird (Camunda hit policies).

ManMap-Mappings und -Optionen

- Formatierte fachliche Nummernkreise: Axelors `<sequence name initial increment padding prefix/>`, beim Speichern neuer Datensätze vergeben, mit BCs Unterscheidung zwischen lückenlosen No. Series (Tabellensperre) und schnellen, lückentoleranten SQL-Sequenzen als Flag `GAPLESS`; RAPs `early`/`late numbering` benennt, wann der Schlüssel vergeben wird (Axelor sequences, BC number sequences, alguidelines No. Series, SAP cheat sheet 36). Das behebt auch die `AUTOID`-Lücke bei MySQL/MariaDB (PR1 P-016).
- `OPTIMISTIC_LOCK`-Bereich über einen Kompositionsbaum (RAP `total etag`) und eine Auswahl der Sperrspalten (iDempiere `set_OptimisticLockingColumns`, OFBiz `enable-lock`, das `lastUpdatedStamp` verlangt) (SAP cheat sheet 36, PO.java, OFBiz Entity Engine Guide).
- Pessimistischer Modifikator `FOR UPDATE [NOWAIT | SKIP LOCKED]` an `CHECKOUT`-Methoden, vom Generator je Dialekt gerendert (Hibernate Introduction); BCs dreistufiges Sperren und `ReadIsolation` pro Lesezugriff zeigen den Isolationsregler pro Anweisung (BC tri-state locking).
- Checker und Generator: Audit-Werte gehören der Plattform, Erstellungsfelder sind unveränderlich und vom Benutzer gesetzte Werte werden überschrieben, wie bei BCs

Systemfeldern; `CREATEDAT`/`CREATEDBY` sollten aus fachlichem Code nicht setzbar sein (BC system fields).

- Option für ein Änderungsprotokoll pro Feld `TRACK`, die Zeilen mit altem und neuem Wert schreibt, wie Odoos `tracking=True` und iDempieres „Maintain Change Log“, mit BCs Warnung, Postentabellen mit hohem Volumen nicht zu verfolgen (Odoo mixins, BC change log).
- Bedingtes Referenzziel per Diskriminator, BCs `TableRelation = if (Type = const (Customer)) Customer else if (Type = const (Item)) Item`, das zugleich als Validierungsmetadaten dient (BC TableRelation).
- Schreibgeschütztes `EntityMapping` auf ein SQL-Select oder eine Datenbank-View mit deklarierten Basistabellen, Hibernates `@Subselect`/`@Synchronize` (Hibernate @Subselect).
- Prädikat auf den Partitionsschlüssel, das generierten UPDATE/DELETE-Anweisungen hinzugefügt wird, Hibernate `@PartitionKey` (Hibernate User Guide §3.11).
- `DEPRECATED` mit den Zuständen Pending/Removed, geprüft wie BCs `ObsoleteState` (Warnung bei Referenzen auf Pending, Fehler bei Removed) (BC ObsoleteState).
- Index-Hinweis für Aggregat-Properties, da BC „passende SIFT-Indizes für alle FlowFields vom Typ Sum oder Count“ verlangt (BC performance).

Repositories und Abfragen

- Optionale Where-Terme mit korrekter `||`-Semantik: Ein Prädikat, dessen Parameter ungesetzt ist, fällt weg, eine Gruppe aus lauter weggefallenen Prädikaten ist neutral statt falsch, und ein vollständig optionales WHERE bei UPDATE/DELETE wird verweigert, nach dem Vorbild von MyBatis Dynamic SQL mit `isEqualToWhenPresent` und `NonRenderingWhereClauseException` (MyBatis Dynamic SQL); das behebt PR2 P-037 und beseitigt den `%`-Workaround.
- Wiederverwendbare, kombinierbare Filterfragmente, einmal in einem Repository deklariert und in mehreren Methoden mit and/or kombiniert, einschließlich Massen-Update/-Delete, wie Spring-Data-Specifications (Spring Data Specifications).
- Generierter AND-Filter-Finder aus einer Feldliste, Axelors `<finder name using="firstName,lastName"/>` (Axelor repositories), und eine Query-by-Example-Methode mit explizitem Matcher, bewusst flach (Spring Data QBE).
- Navigation mit `collect`/`collectChildren` in gemappten Abfragen („alle Geschäftspartner der getroffenen Aufträge“, „Zeilen der getroffenen Lieferungen“) als deklarativer Join-Ersatz ohne Lazy Loading (IQueryBuilder.java).
- Strategieschalter für `listJoin`: JOIN gegenüber gebündelter IN-Liste gegenüber Subselect pro Methode, wobei die IN-Listen-Variante die Zeilenvervielfachung bei breiten Elternzeilen vermeidet (Hibernate Introduction).
- Arten von Ergebnisfenstern `page` (Count-Abfrage), `slice` (Probe mit Größe+1) und `keyset` mit Regeln zur Bearbeitungszeit (nicht-null Sortierschlüssel, angehängter Schlüssel, kein Offset) (Spring scrolling, jOOQ seek blog).
- Checker-Warnung für einen Repository-Aufruf innerhalb einer Schleife über Entities, die die N+1-Gestalt sichtbar macht, so wie Governor Limits in Salesforce Bulk-Muster erzwingen (Trailhead bulk triggers).
- Checker: Ein Methodenrumpf berührt eine Referenz, die nicht in seinem deklarierten Load-Graphen ist; Jmix' „cannot get unfetched attribute“ wird in die Bearbeitungszeit verlegt (Jmix fetching).

Batchjobs, Events und Prozesse

- Retry-Strategie als wiederverwendbare Deklaration mit den Feldern von Temporal (Anfangsintervall, Backoff, maximales Intervall, maximale Versuche, nicht wiederholbare Problemtypen, Gesamt-Timeout), anhängbar an Consumer, Event-Handler und Prozessschritte (Temporal retry policies).
- `INCIDENT` als terminale Reaktion in der Exception-Strategie, für den Betrieb sichtbar und behebbar, nach Camunda (Camunda job workers); Dead-Letter als der andere Ort, an den fehlgeschlagene Arbeit gehen kann (Axon event processors).
- Zeitplanoptionen am `consumer/producer pair`: Überlappungsstrategie (überspringen, puffern, zulassen), Nachholfenster, Timeout pro Element, deklariertes „läuft nie gleichzeitig“ und „muss wiederholbar sein“ (Temporal schedules, ODC timers).
- `idempotent by <key>` an Commands und Service Methods mit Stripe-Semantik (erstes Ergebnis speichern, bei Retry wiedergeben, abweichende Parameter zurückweisen, begrenzte Aufbewahrung) (Stripe idempotent requests); Prozessstart-Strategien `reject duplicate | use existing` (Temporal workflow id).
- Handler-Option `isolated`, die den Handler in einer eigenen Transaktion ausführt, sodass sein Fehlschlag den Publisher nicht zurückrollen kann, BCs isolierte Events (BC isolated events).
- Event-Versionierung: Sobald Events in einer Registry oder Outbox persistiert sind, liest neuer Code alte serialisierte Events, daher braucht es eine `revision` an Event-Typen und einen Platz für einen Upcaster; Identifikatoren und Zeitstempel bleiben unveränderlich (Axon event versioning).
- Ein Schutz „nicht während Replay“ für Handler mit Seiteneffekten, falls Events jemals wieder abgespielt werden, Axons `isLive()` (Axon aggregate).
- Test-DSL: `run command` sichert publizierte Events zu (`expect event RechnungFreigegeben with rechnungKey = ...`), Spring Modulith' `AssertablePublishedEvents` (Modulith events).
- Versionsbindung für Regeltabellen und Formulare, die von laufenden Prozessen verwendet werden (`latest | deployment | versionTag`) (Camunda business rule tasks).
- Generierter Modul-Steckbrief pro Bereich (Entities, Commands, Services, publizierte und konsumierte Events, Konfiguration), Spring Modulith' `writeModuleCanvases()` (Modulith documentation).

Rollen, Konfiguration und Tests

- Lese-/Schreibrecht pro Rolle auf Member-Ebene, einmal an Business Properties deklariert und in den schreibgeschützten Zustand der UI und das Weglassen von Feldern in der API abgeleitet, wie Mendix-Zugriffsregeln und Causeways `hidden(Where)`/`editing` (Mendix access rules, Causeway @Property); das ist ein eigenes Thema, würde aber die Grobheit von `CAN_OPEN_RO` beheben (PR2 P-060).
- Checker-Regeln nach dem Metamodell-Validator von Causeway: Ein Command-Parameter vom Listentyp braucht eine deklarierte Auswahlquelle, und eine `optional`-Property muss mit ihrem `NOTNULL`-Mapping übereinstimmen (Causeway validator config).
- Command-Log-Strategie `only-if-system-changed`, wenn Commands aufgezeichnet werden, sodass sichere Suchen übersprungen werden (Causeway command log).
- Beigesteuerte Member im Mixin-Stil: eine Action oder abgeleitete Property, die einer Entity aus einem anderen Modell beigesteuert wird, ohne sie zu bearbeiten, Causeway-Mixins; nützlich, falls MoWare je modulexterne Erweiterung braucht, sonst niedrige Priorität (Causeway mixins). Erweiterbare Enums und Tabellenerweiterungen (BC `Extensible = true`, `enumextension`, `tableextension`) lösen ein Problem des ISV-Ökosystems, das MoWare nicht hat, und werden nicht empfohlen (BC extensible enums).

- Checker-Regeln im Stil des RAP-Strict-Mode für ObjectFlow: kein `CHECKIN/DELETE/STATEMENT` innerhalb einer Determination oder Validation, kein `emit` außerhalb von `FINAL OK_CONCLUSION` oder der Speicherphase und Zyklenerkennung über Feldauslöser (SAP cheat sheet 08).
- Rein additive API-Evolution mit `deprecated`-Flags statt versionierter Endpunkte, GraphQLs versionslose Praxis (GraphQL schema design).
- Sperrlebensdauer und Ablauf im Draft-Stil als Konfigurationswerte, falls B7 verfolgt wird: RAP verwendet eine exklusive Sperre von 15 Minuten und einen Draft-Ablauf nach 28 Tagen, deren Konfigurierbarkeit nicht verifiziert ist (SAP Learning draft).